

3. ESTRATEGIAS DE BÚSQUEDA INFORMADAS (PARTE 1)

IA 3.2 - Programación III

2° C - 2025

Dr. Mauro Lucci



Repaso

Las estrategias de búsqueda **no informadas** usan únicamente la definición del problema de búsqueda. Todo lo que hacen es expandir nodos siguiendo algún criterio preestablecido y distinguir estados objetivos de no objetivos.

¿Cómo podemos guiar la búsqueda para encontrar soluciones de forma más inteligente?

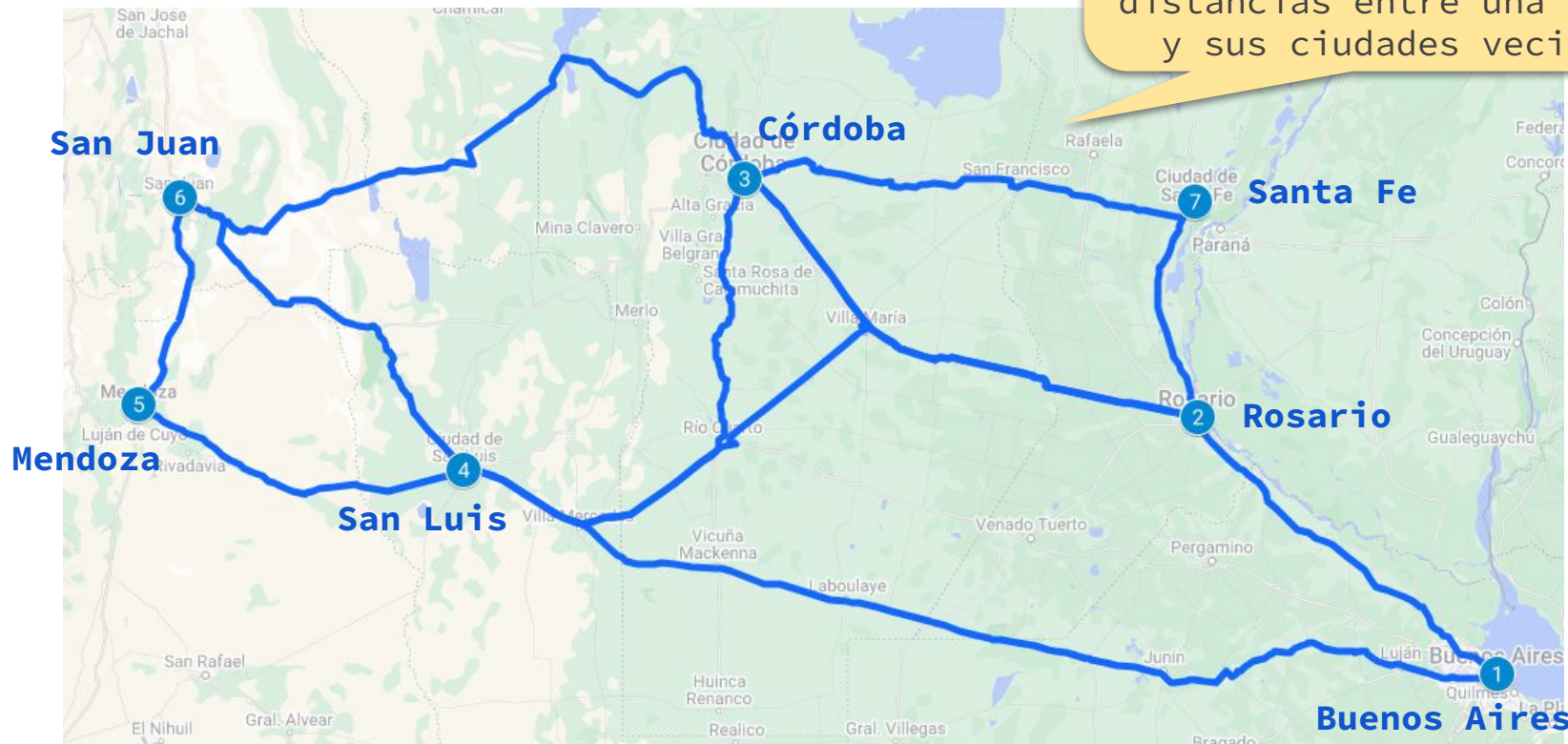
Veremos estrategias de búsqueda **informadas**.

Motivación

Problema de camino más corto

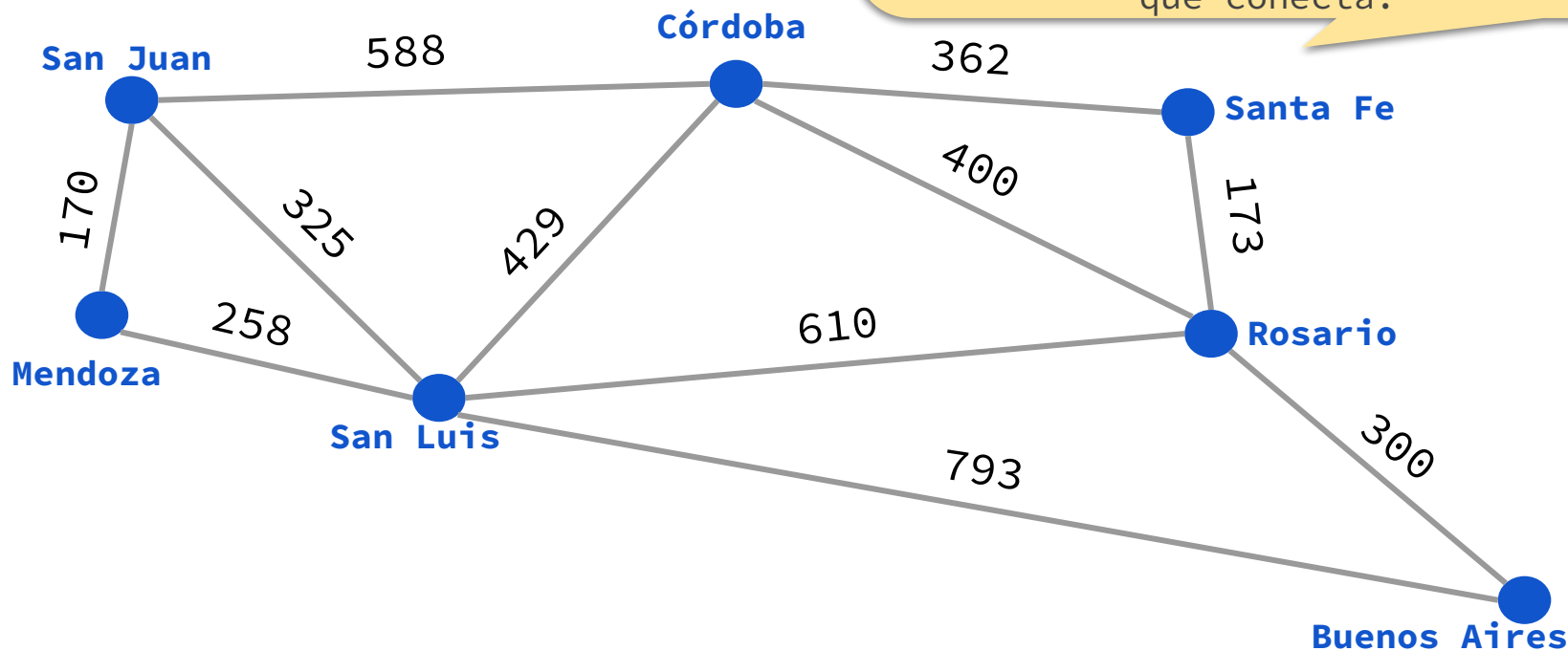
— — —

Supuesto. No conocemos las distancias entre todo par de ciudades. Sino que conocemos únicamente las distancias entre una ciudad y sus ciudades vecinas.



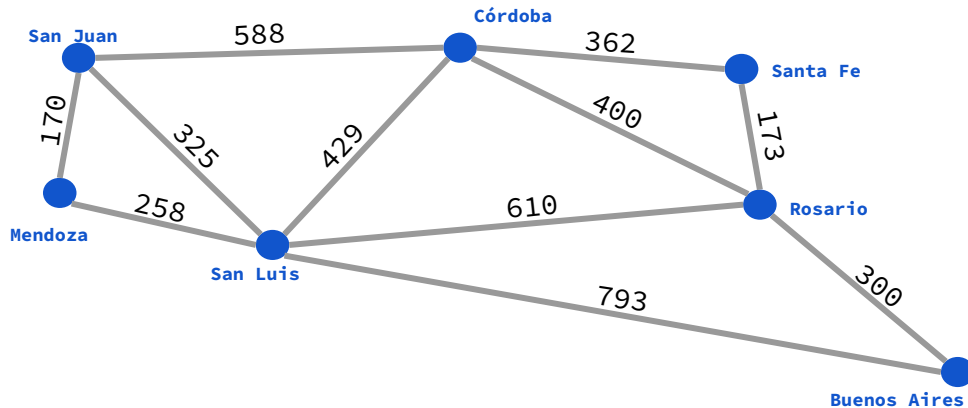
Problema de camino más corto

Los datos de entrada de este problema se suelen representar con un **grafo**. Los vértices son las ciudades. Las aristas conectan ciudades vecinas. El costo de cada arista es la distancia en kilómetros entre las dos ciudades que conecta.



Descripción

— — —



Objetivo. Encontrar el camino más corto de Buenos Aires a San Juan.

Reglas. Las acciones consisten en viajar de la ciudad actual a otra vecina, es decir, conectadas con una arista.



Ejemplo

— — —



Costo del camino: $300 + 400 + 588 = 1288$



Propuesta de formulación

Notación:

- C : conjunto de ciudades.
- $N(c)$: conjunto de ciudades vecinas a $c \in C$.
- $\text{dist}(c_1, c_2)$: distancia entre $c_1, c_2 \in C$.

En esta formulación, un **estado** se representa con una ciudad c de C que indica la ciudad actual. En total hay $|C|$ estados.

Una **acción** se representa con una ciudad c' de C que indica viajar de la ciudad actual a c' .



Ejemplo

— — —



Nuestra representación:





Propuesta de formulación

— — —

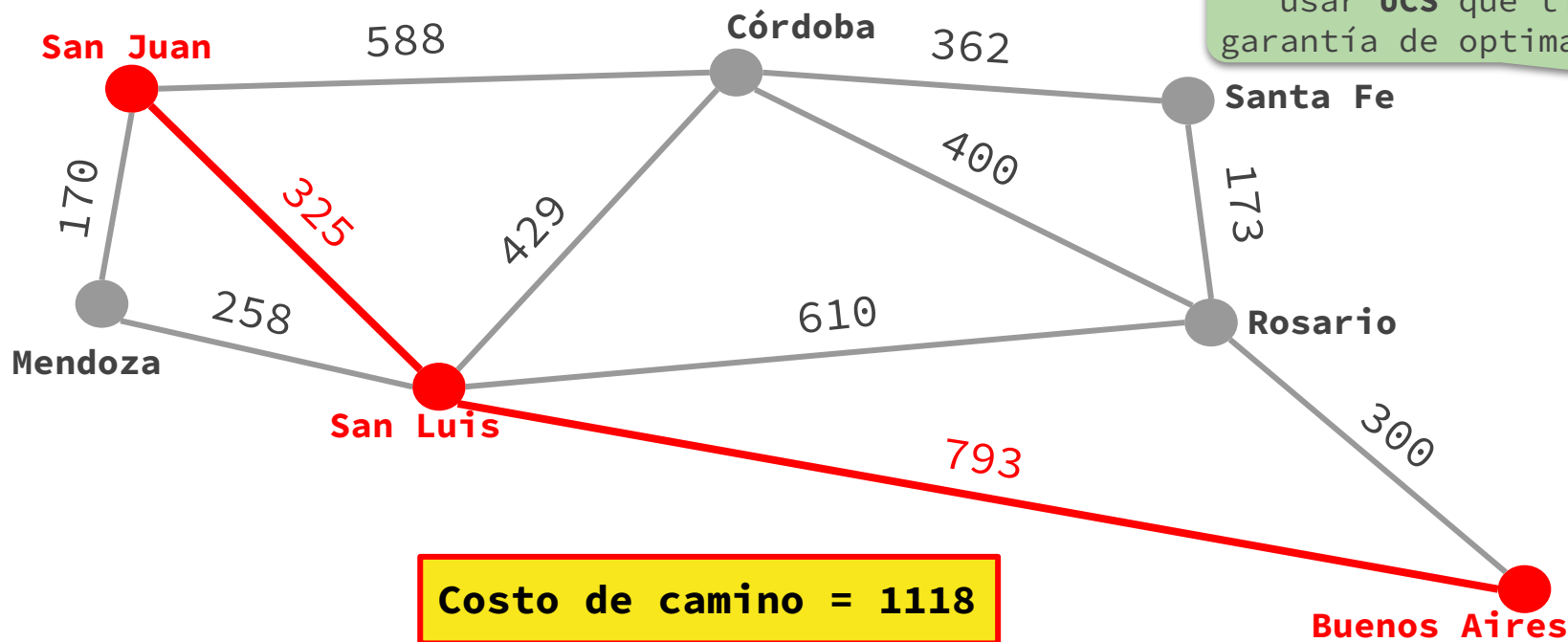
- **Estado inicial.** “Buenos Aires”.
- **Acciones.** Dado un estado c ,
 $ACCIONES(c) = N(c)$.
- **Modelo de transiciones.** Dado un estado c y una acción c'
 $RESULTADO(c, c') = c'$.
- **Test objetivo.** Dado un estado c , $TEST-OBJETIVO(c) = (c == \text{“San Juan”})$
- **Costo de camino.** El costo de camino es la suma de los costos individuales.
Dado un estado c y una acción c' , el costo individual:
 $COSTO-INDIVIDUAL(c, c') = \text{dist}(c, c')$.



Solución óptima

Podemos resolver este mapa particular con un **algoritmo de camino más corto**, e.g. Dijkstra... Pero como dijimos la primera clase, nos interesa resolver problemas de búsqueda cuyo grafo de espacio de estados puede ser **demasiado grande** para ser almacenado por completo en memoria.

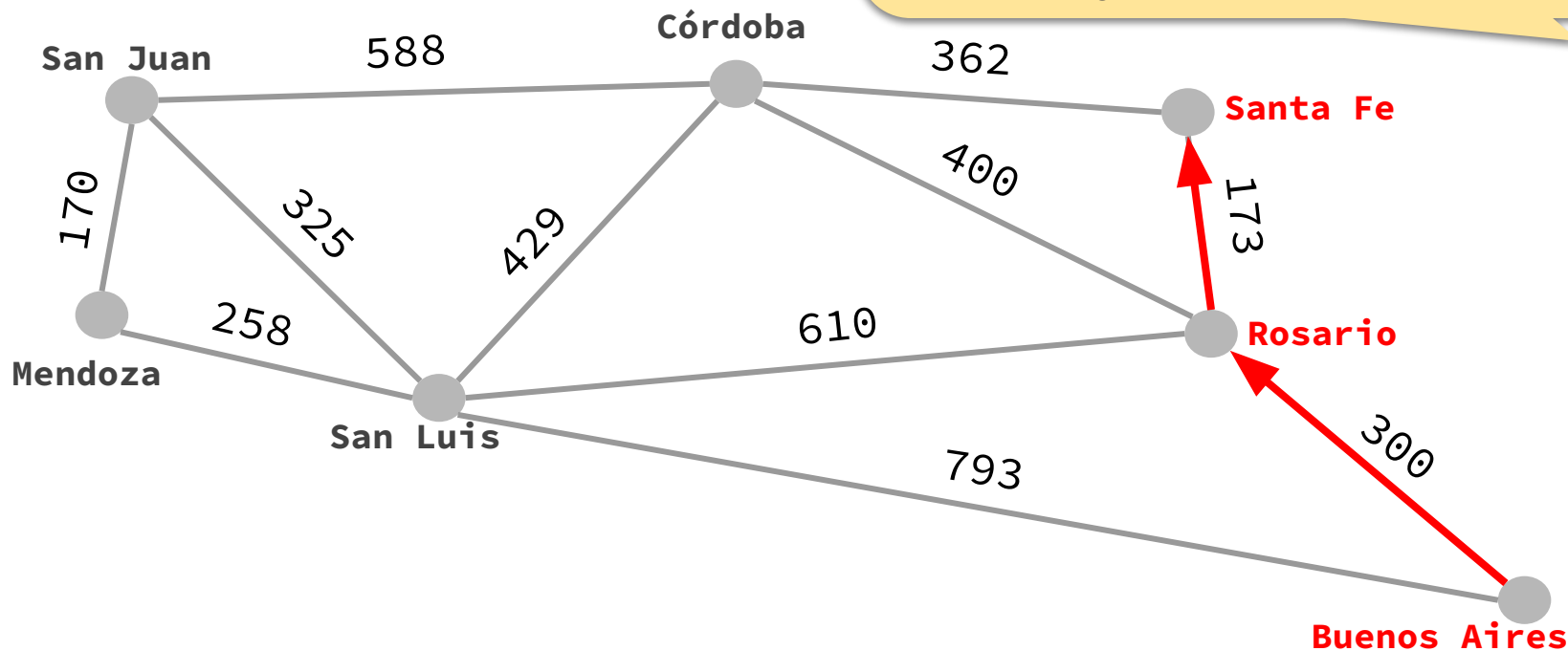
En ese caso, podemos usar **UCS** que tiene garantía de optimalidad.



Problema de camino más corto

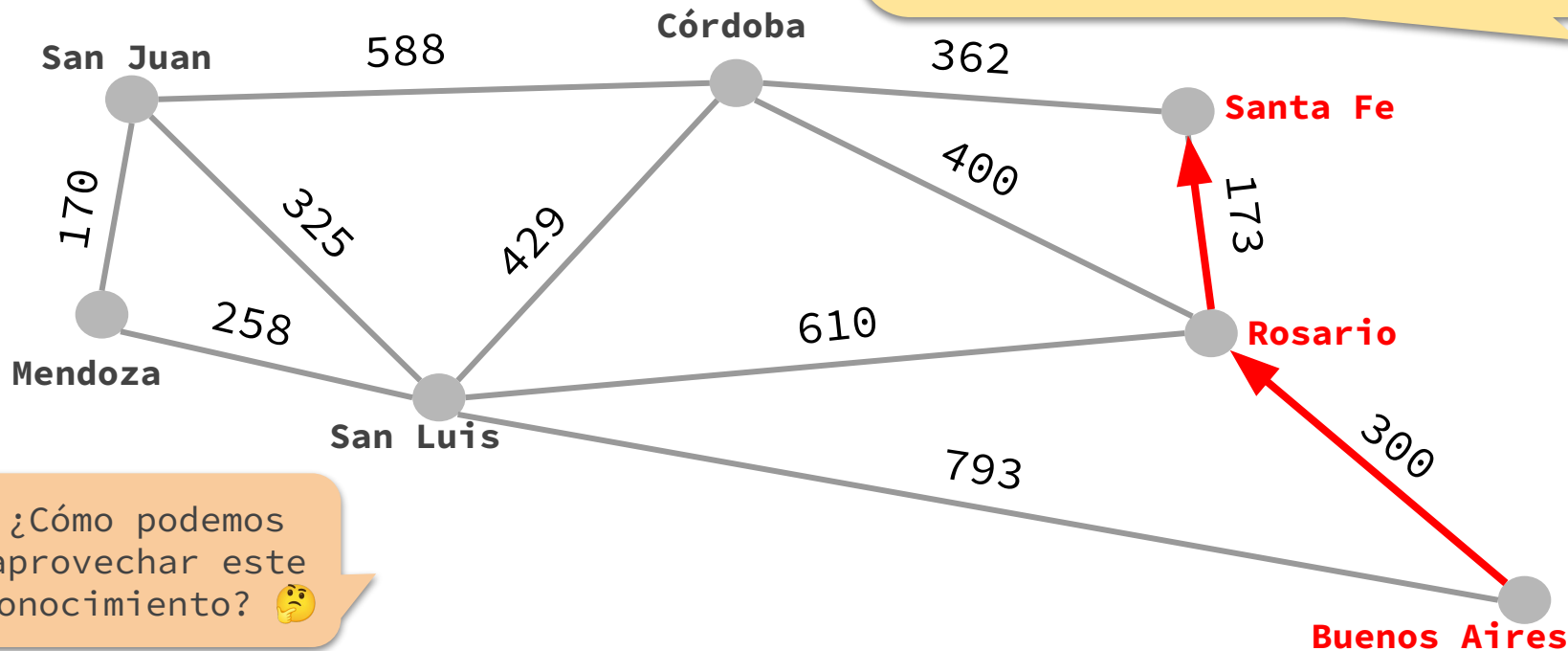
Pero UCS no es “tan inteligente”.

Por ejemplo, comienza expandiendo “Bs. As.” → “Rosario” → “Santa Fe” → ... Pero es claro que viajar a “Santa Fe” no es la mejor opción para nuestro objetivo de llegar a “San Juan”, sino que es más directo viajar a “Córdoba” o “San Luis”.



Problema de camino más corto

Pero UCS no es “tan inteligente”.
Por ejemplo, comienza expandiendo
“Bs. As.” → “Rosario” → “Santa Fe” → ...
Pero es claro que viajar a “Santa Fe” no
es la mejor opción para nuestro objetivo
de llegar a “San Juan”, sino que es más
directo viajar a “Córdoba” o “San Luis”.



¿Cómo podemos
aprovechar este
conocimiento? 🤔

Estrategias de búsqueda **informadas**

— — —

- Saben si un nodo tiene un estado **más prometedor** que otro y lo expanden antes.
- Además de la definición del problema de búsqueda, requieren cierto **conocimiento específico** del problema.
- Incorporan una **función heurística** h donde, para cada nodo n ,
 $h(n)$: es una estimación del menor costo de camino desde el estado de n a un estado objetivo.
- Si n es un nodo con un estado objetivo, es necesario que h verifique $h(n) = 0$.



Ejemplo – Problema de camino más corto

— — —

La intuición de estar más cerca de “San Juan” la podemos capturar con una función que mida **distancia en línea recta**.



Ejemplo. distancia en línea recta entre Santa Fe y San Juan.

Distancia en línea
recta a San Juan
desde:

Buenos Aires	= 997
Rosario	= 747
Santa Fe	= 743
Córdoba	= 417
San Luis	= 284
Mendoza	= 148
San Juan	= 0



Ejemplo – Problema de camino más corto

La intuición de estar más cerca de “San Juan” la podemos capturar con una función que mida **distancia en línea recta**.



Volviendo al problema... Luego de expandir “Bs. As.”, a la hora de elegir entre expandir “Rosario” o “San Luis”, lo intuitivo es seguir con “San Luis” pues nos acercará más a nuestro objetivo de llegar a “San Juan” (747 km vs. 284 km).

Distancia en línea recta a San Juan desde:

Buenos Aires	= 997
Rosario	= 747
Santa Fe	= 743
Córdoba	= 417
San Luis	= 284
Mendoza	= 148
San Juan	= 0

Búsqueda avara primero el mejor

Greedy Best-First Search
(GBFS)

Intenta encontrar
rápidamente una solución.

Los nodos de la frontera se expanden según su cercanía a un nodo objetivo.

Dada una **función heurística** h , se expande siempre el nodo n de la frontera con el **menor valor heurístico**.

— — —



Ejemplo – Problema de camino más corto

— — —

Vamos a resolver el problema de camino más corto con la búsqueda de árbol usando la estrategia de búsqueda avara primero el mejor.



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1			
2			
3			

Anotaremos el valor
heurístico arriba del
nodo en color **violeta**.

997

n₀ :
Bs. As.
0



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			

Extraemos el único nodo de la frontera y lo expandimos.

997

n_0 :
Bs. As.
0



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			

997

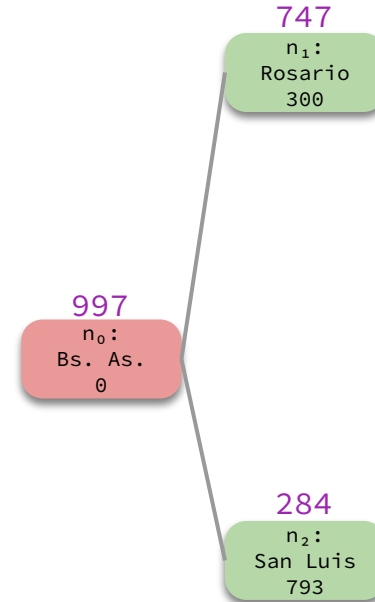
$n_0 :$
Bs. As.
0



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			

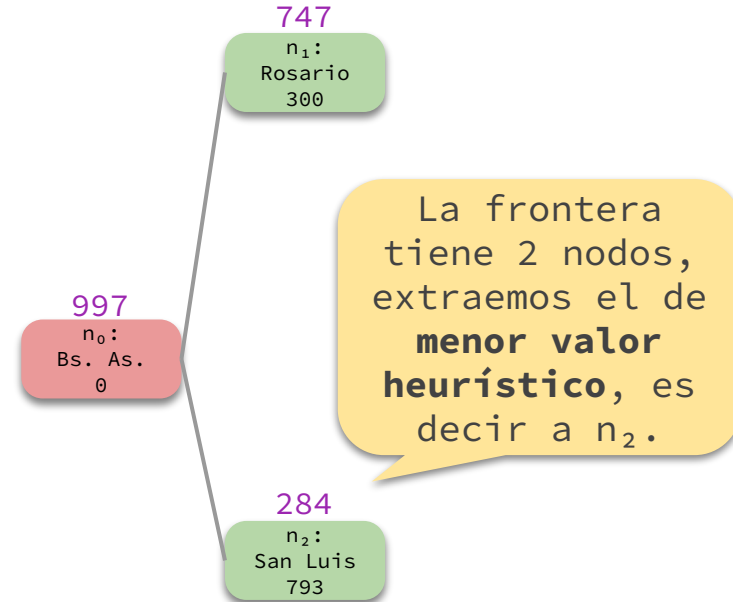




Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			

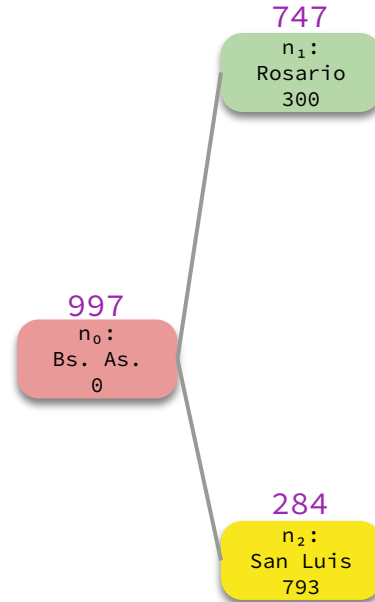




Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	
3			

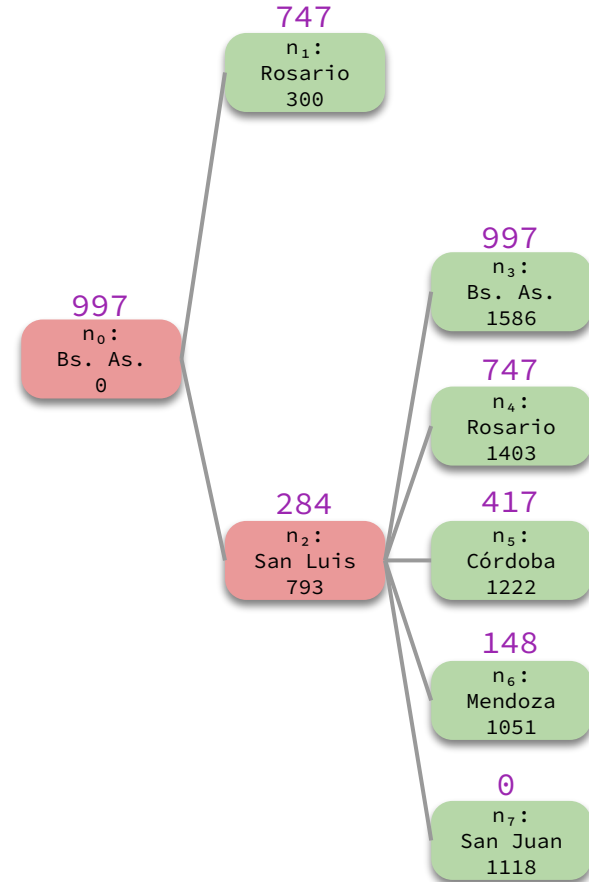




Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3			



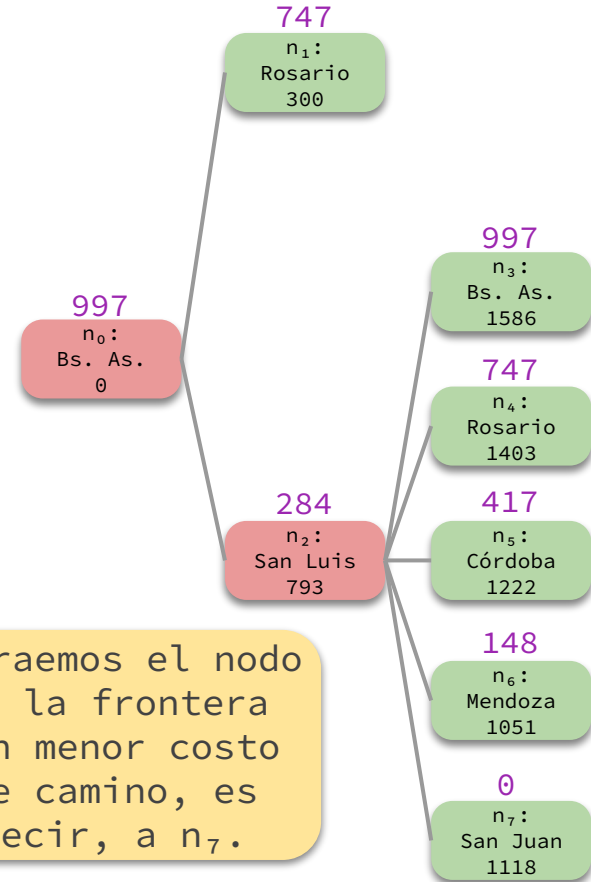


Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3			

Extraemos el nodo de la frontera con menor costo de camino, es decir, a n_7 .



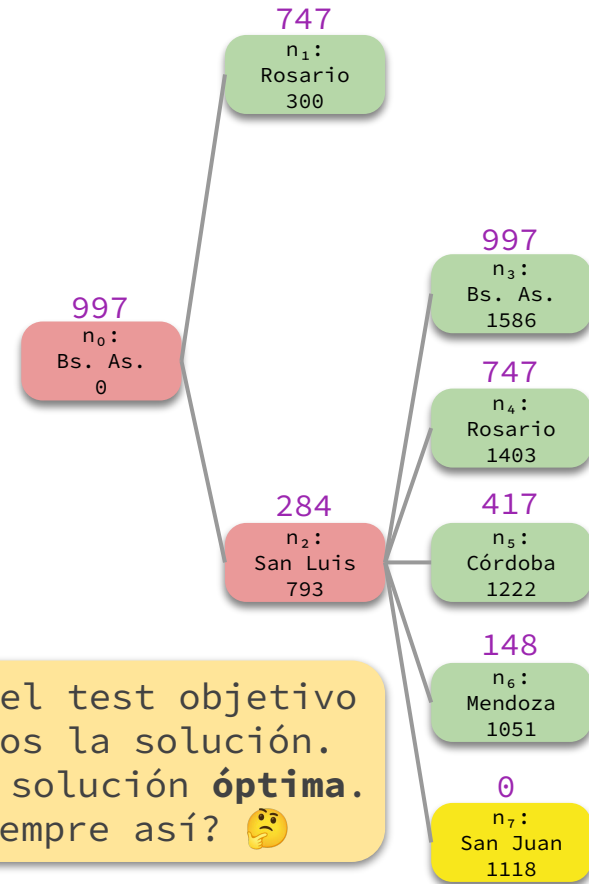


Ejemplo

— — —

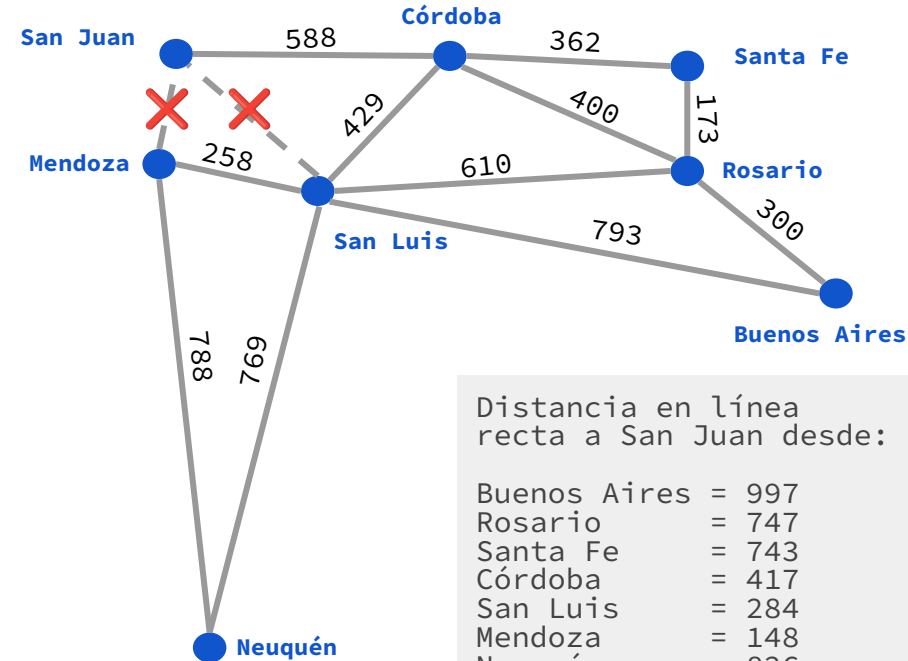
Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3	n_7	$\{n_1, n_3, n_4, n_5, n_6\}$	¡FIN!

Ejecutamos el test objetivo y retornamos la solución. Encontramos solución **óptima**. ¿Será siempre así? 🤔





Ejemplo más interesante



Supongamos que se cortan las rutas que conectan a Mendoza y San Luis con San Juan y que se agrega Neuquén con conexiones a Mendoza y San Luis.

Es decir, el problema ahora es encontrar el camino más corto de Bs. As. a San Juan en el mapa de la izquierda.

Vamos a resolver este problema nuevamente con la búsqueda de árbol usando la estrategia de búsqueda avara primero el mejor.



Ejemplo

— — —

997

n_0 :
Bs. As.
 $\emptyset$

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			



Ejemplo

— — —

997

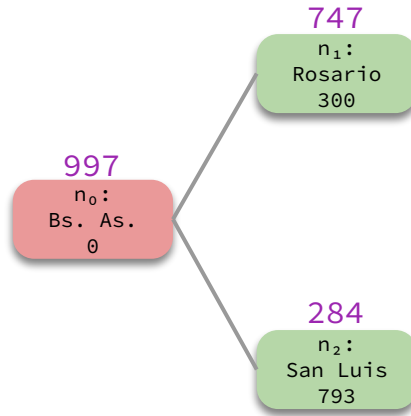
n_0 :
Bs. As.
 $\emptyset$

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			
5			



Ejemplo

— — —

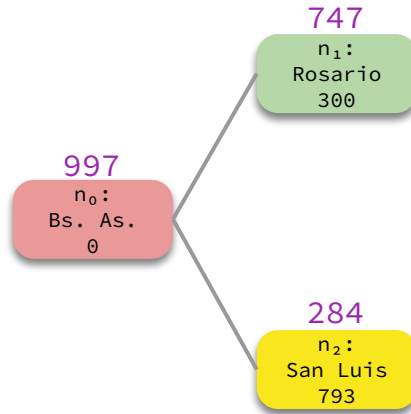


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{ }	{n ₁ , n ₂ }
2			
3			
4			
5			



Ejemplo

— — —

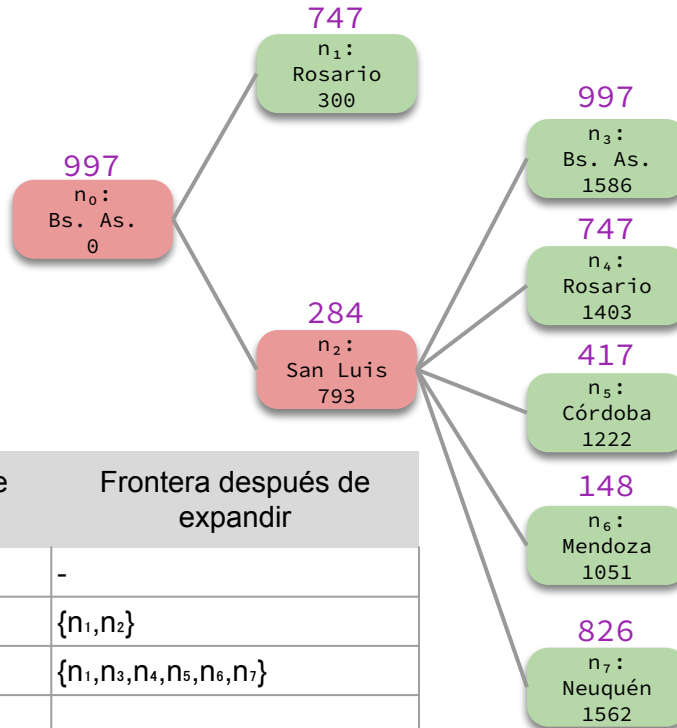


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	
3			
4			
5			



Ejemplo

— — —

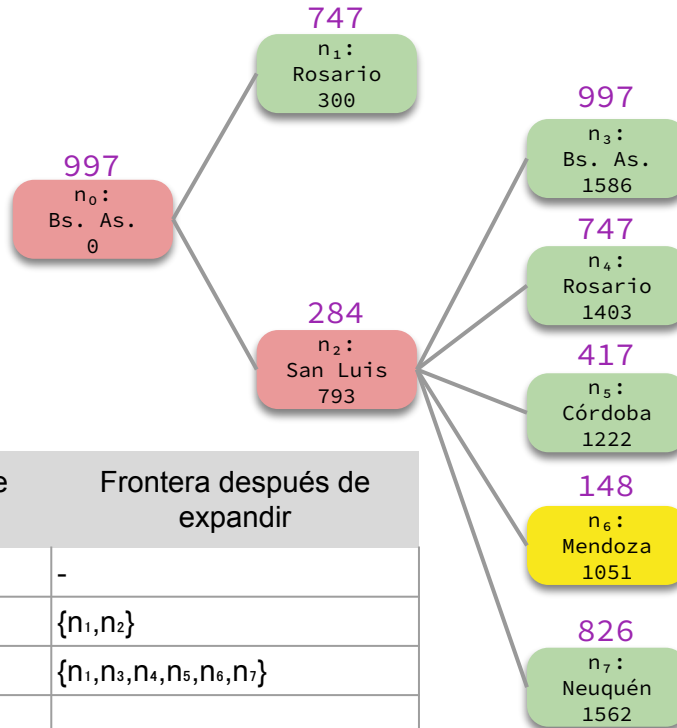


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₆ , n ₇ }
3			
4			
5			



Ejemplo

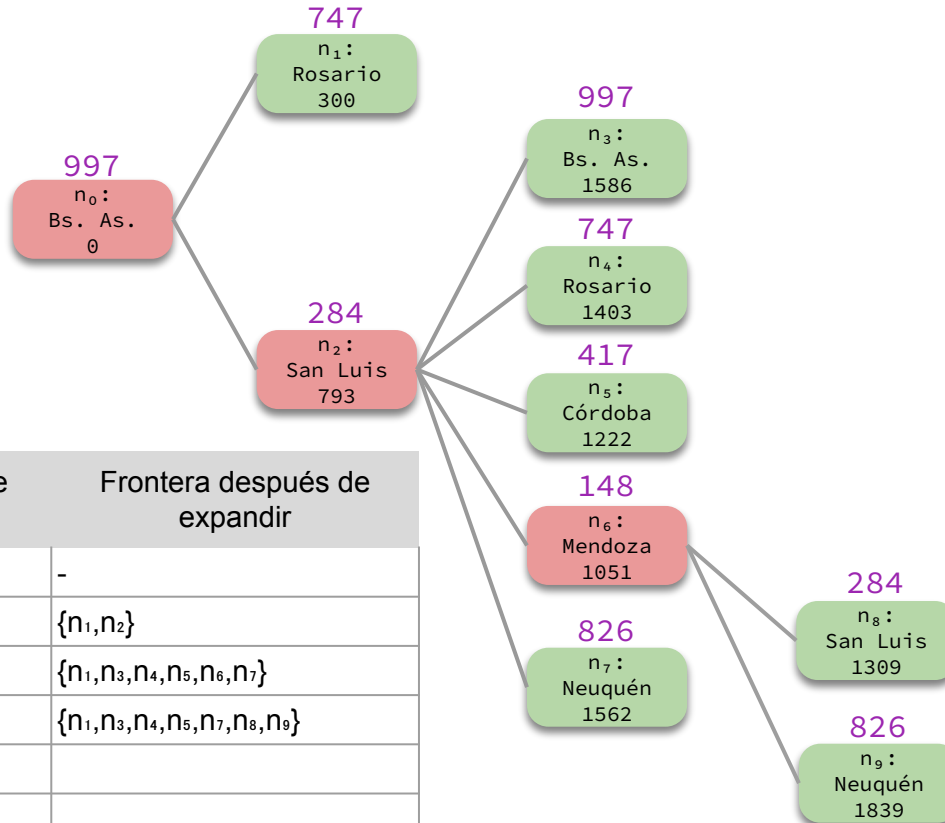
— — —



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₆ , n ₇ }
3	n ₆	{n ₁ , n ₃ , n ₄ , n ₅ , n ₇ }	
4			
5			



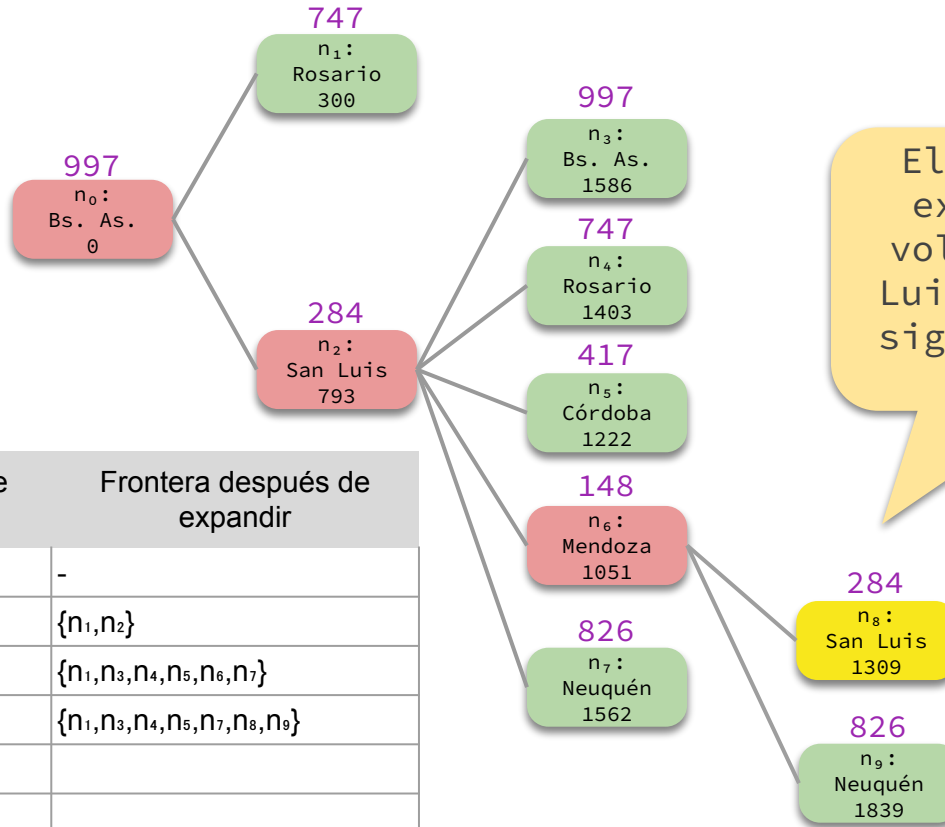
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₆ , n ₇ }
3	n ₆	{n ₁ , n ₃ , n ₄ , n ₅ , n ₇ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₇ , n ₈ , n ₉ }
4			
5			



Ejemplo



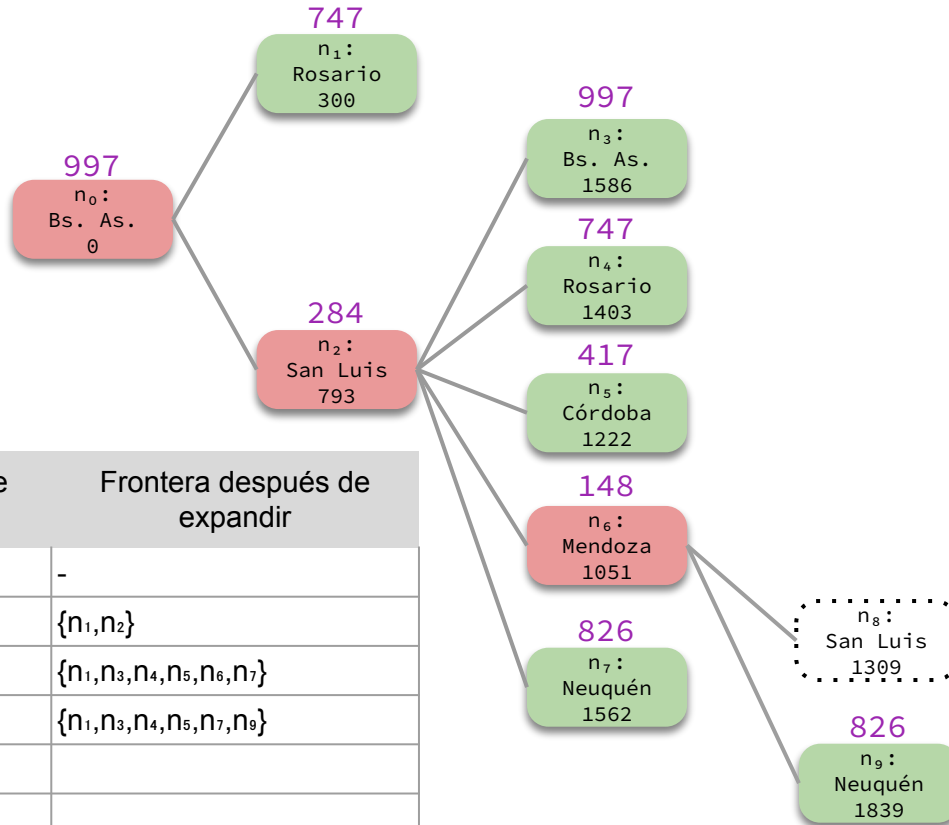
El próximo nodo a expandir es n_8 y volveríamos a “San Luis”... ¡Estaríamos siguiendo un **camino cíclico**!

Supongamos que agregamos la detección de ciclos.

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3	n_6	$\{n_1, n_3, n_4, n_5, n_7\}$	$\{n_1, n_3, n_4, n_5, n_7, n_8, n_9\}$
4	n_8	$\{n_1, n_3, n_4, n_5, n_7, n_9\}$	
5			



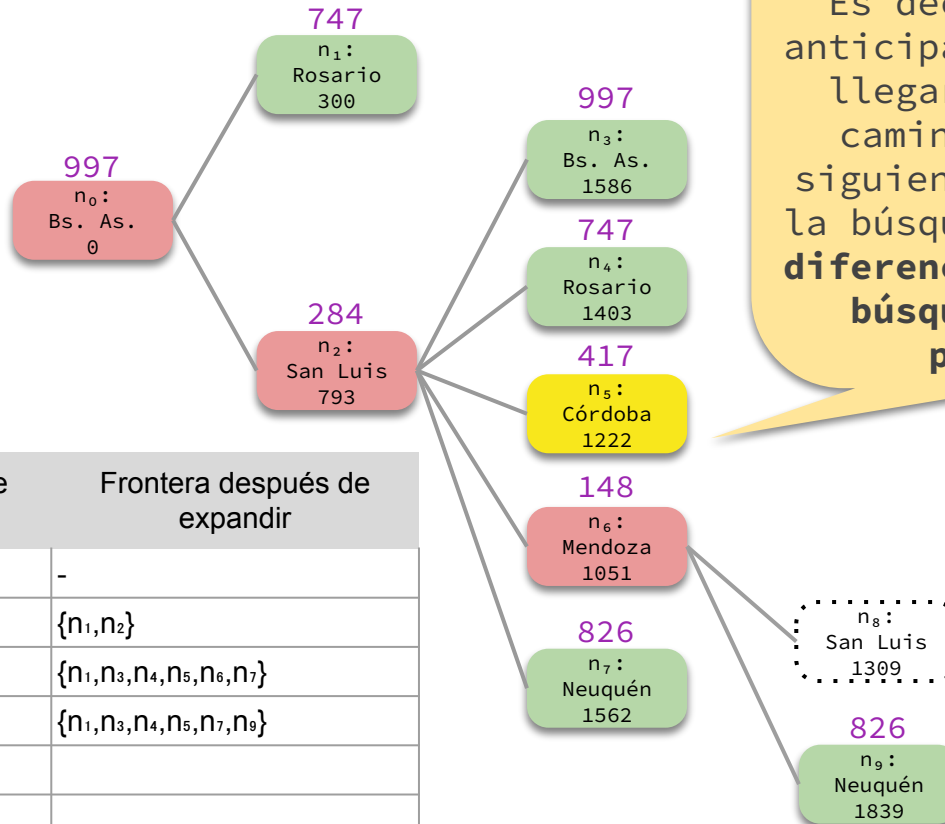
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3	n_6	$\{n_1, n_3, n_4, n_5, n_7\}$	$\{n_1, n_3, n_4, n_5, n_7, n_9\}$
4			
5			



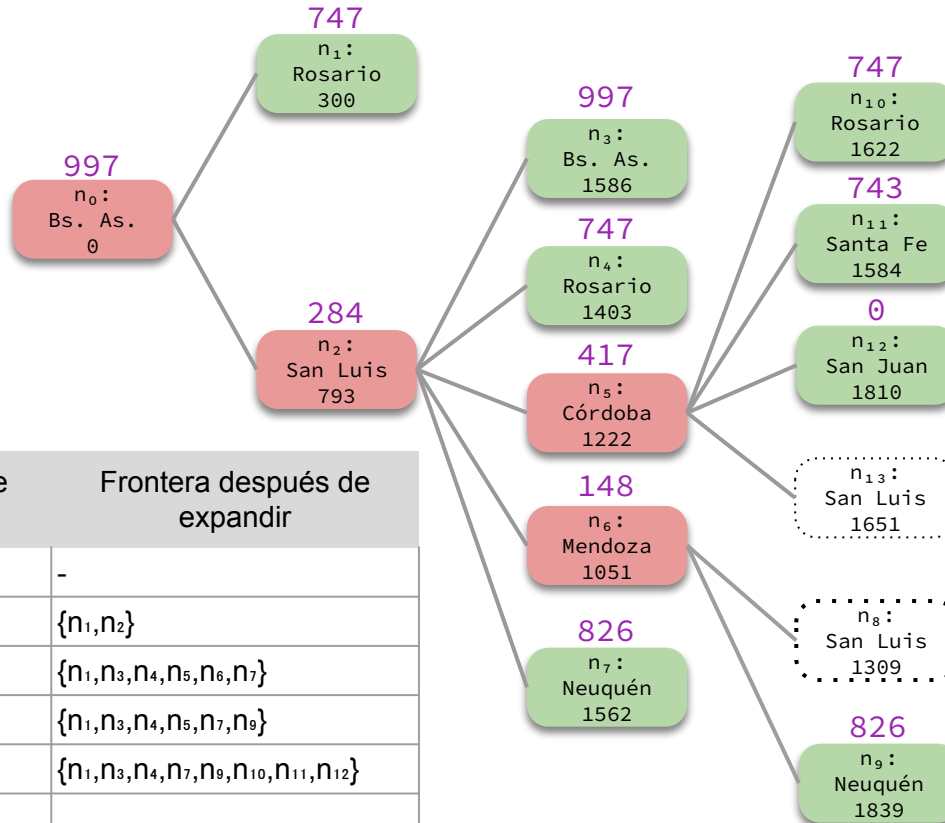
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3	n_6	$\{n_1, n_3, n_4, n_5, n_7\}$	$\{n_1, n_3, n_4, n_5, n_7, n_9\}$
4	n_5	$\{n_1, n_3, n_4, n_7, n_9\}$	
5			



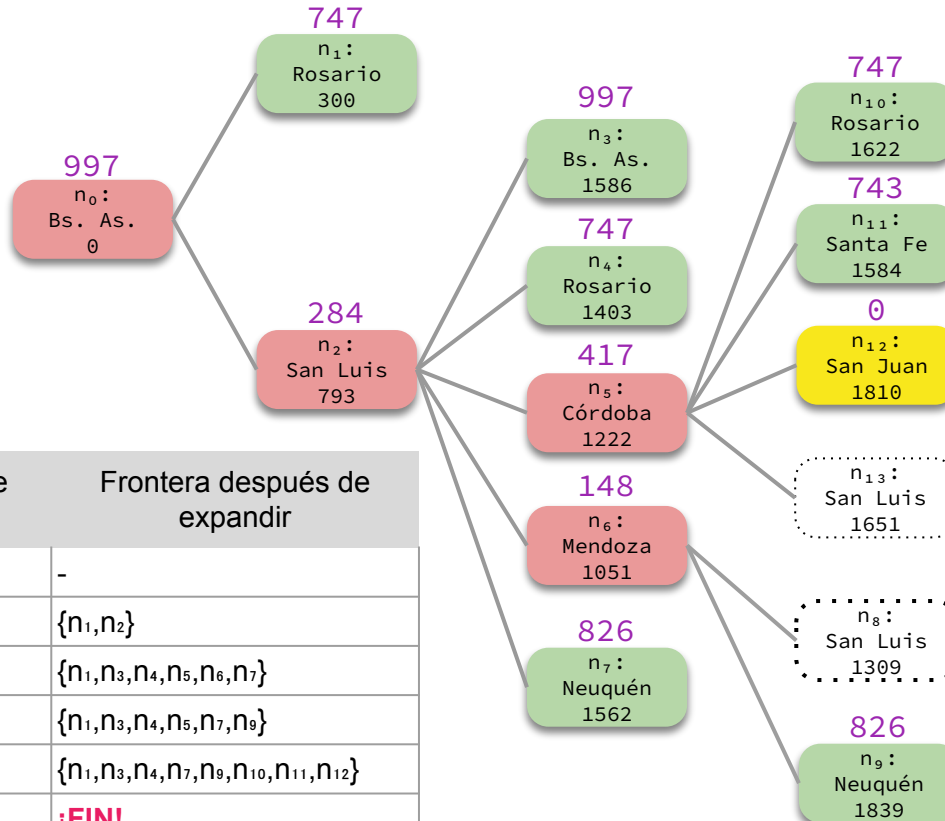
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₆ , n ₇ }
3	n ₆	{n ₁ , n ₃ , n ₄ , n ₅ , n ₇ }	{n ₁ , n ₃ , n ₄ , n ₅ , n ₇ , n ₉ }
4	n ₅	{n ₁ , n ₃ , n ₄ , n ₇ , n ₉ }	{n ₁ , n ₃ , n ₄ , n ₇ , n ₉ , n ₁₀ , n ₁₁ , n ₁₂ }
5			



Ejemplo



Encontramos una solución con costo 1810. Esta solución **no es óptima**, dado que existe el camino no explorado “Bs. As.” → “Rosario” → “Córdoba” → “San Juan” con costo 1288.

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3, n_4, n_5, n_6, n_7\}$
3	n_6	$\{n_1, n_3, n_4, n_5, n_7\}$	$\{n_1, n_3, n_4, n_5, n_7, n_9\}$
4	n_5	$\{n_1, n_3, n_4, n_7, n_9\}$	$\{n_1, n_3, n_4, n_7, n_9, n_{10}, n_{11}, n_{12}\}$
5	n_{12}	$\{n_1, n_3, n_4, n_7, n_9, n_{10}, n_{11}\}$	¡FIN!

Observaciones

De estos ejemplos podemos observar que la búsqueda de árbol con estrategia GBFS:

1. Puede seguir caminos cíclicos. Es necesario detectarlos para garantizar **completitud** en espacios de estados finitos.
2. Tiende a seguir caminos en profundidad. Pero a diferencia de DFS, puede **retroceder** antes de llegar a una hoja si el camino se empieza a alejar del objetivo.
3. No tiene garantía de **optimalidad**.



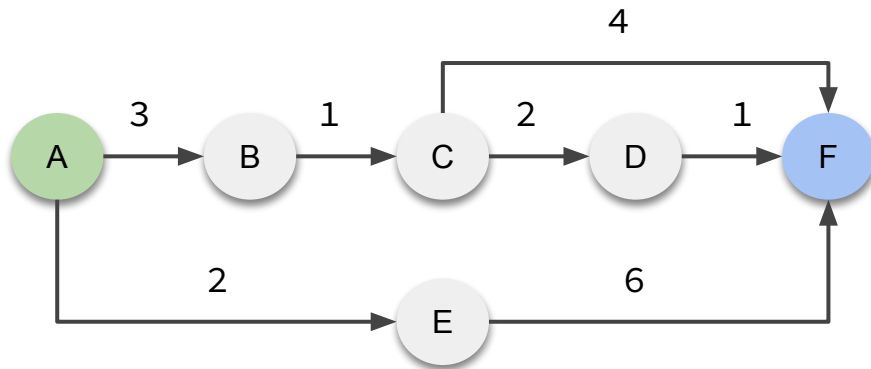
Ejercicio

Sea un problema con el siguiente grafo de espacio de estados, donde A es el estado inicial y F es el estado objetivo.

Resolverlo con el algoritmo general de búsqueda de árbol con la estrategia GBFS.

Usar la heurística dada por:

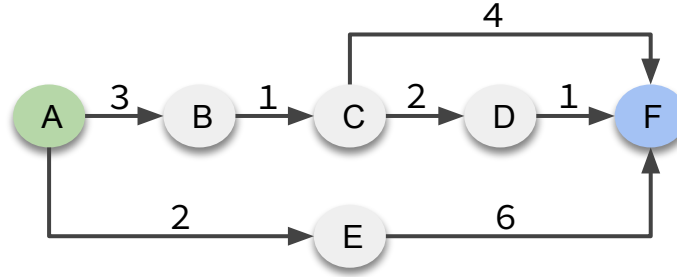
A = 6, B = 3, C = 2, D = 1, E = 5, F = 0





Respuesta

— — —



Valores heurísticos:

A = 6, B = 3, C = 2,
D = 1, E = 5, F = 0

6 n_0
A 0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1			
2			
3			
4			



Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			

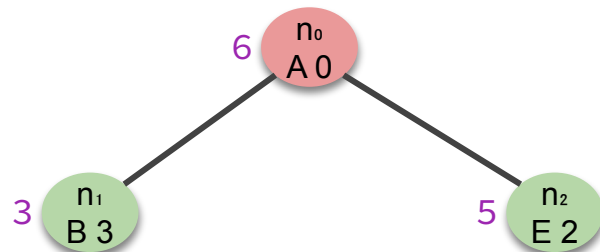
6 $\begin{matrix} n_0 \\ A 0 \end{matrix}$



Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			
4			

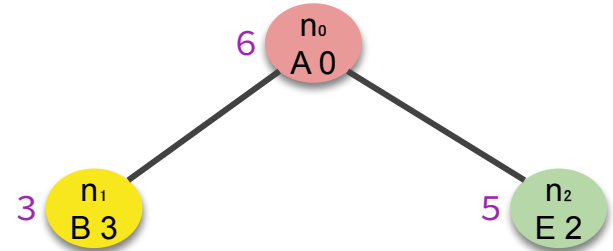




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	
3			
4			

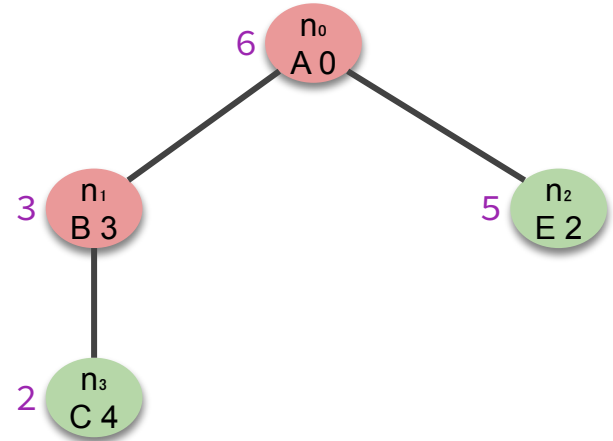




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3			
4			

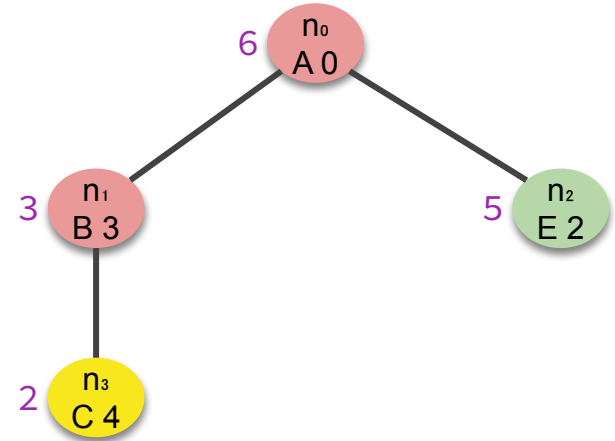




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	
4			

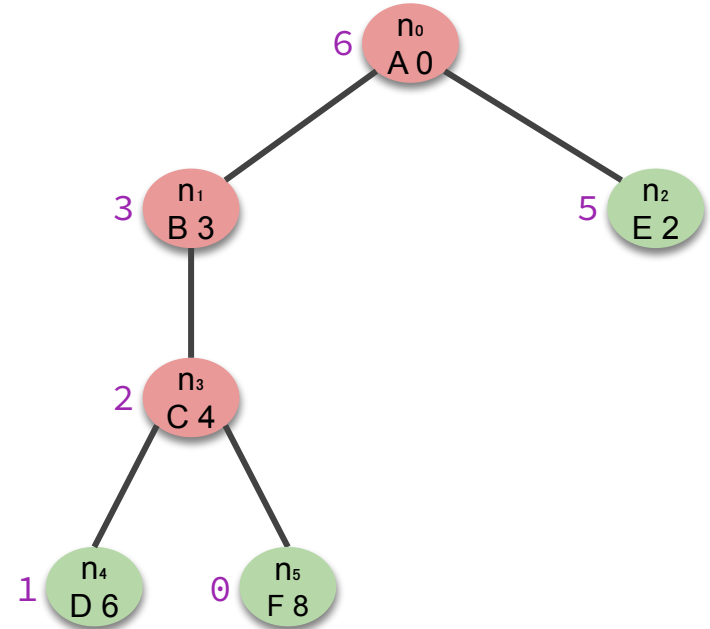




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4			

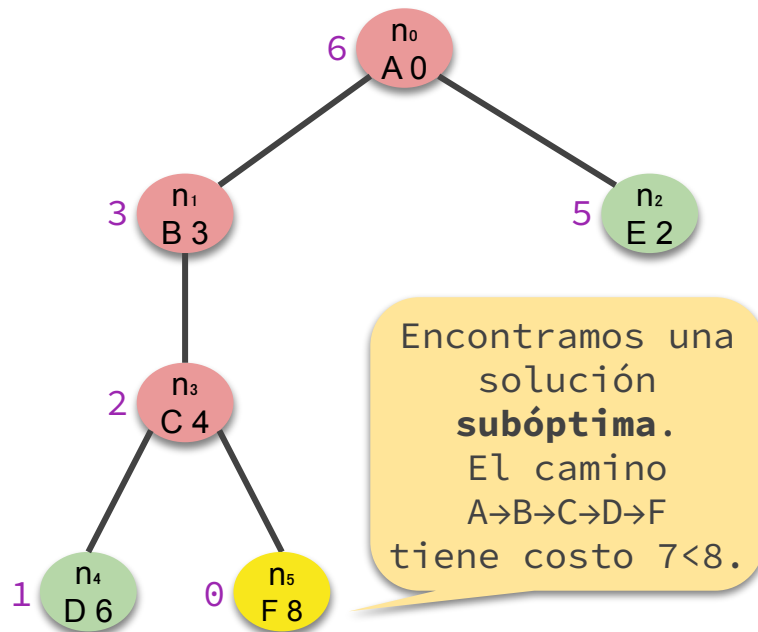




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4	n_5	$\{n_2, n_4\}$	¡FIN!



Implementación

- ¿Cómo elegimos de la frontera el próximo nodo a expandir?

El nodo n con el menor costo $h(n)$.

- ¿Cómo lo logramos?

Al igual que en UCS, usando el TAD **cola de prioridad** para la frontera pero ordenando los nodos por la función heurística.



Algoritmo GBFS de grafo

```
1 function GRAPH-GBFS(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   frontera ← ColaPrioridad()
4   frontera.encolar(n0, problema.h(n0))
5   alcanzados ← {n0.estado: n0.costo}
6   do
7     if frontera.vací() then return fallo
8     n ← frontera.desencolar()
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      c' ← n.costos + problema.costos-individual(n.estado, a)
13      if s' is not in alcanzados or c' < alcanzados[s'] then
14        n' ← Nodo(s', n, a, c')
15        alcanzados[s'] ← c'
16        frontera.encolar(n', problema.h(n'))
```

Suponemos que el problema tiene un **método** *h* que implementa la heurística.

TREE-GBFS es análogo. Hay que borrar únicamente las partes relacionadas con el diccionario de alcanzados y agregar la detección de ciclos.

Performance de TREE-GBFS

— — —

- **Completitud de TREE-GBFS.**

✗ Si el espacio de estados es infinito o es finito pero no se detectan ciclos.

✓ Si el espacio de estados es finito y se detectan ciclos.

- **Optimalidad.** ✗

Performance de TREE-GBFS

En el peor caso, la heurística podría ser

$$h(n) = 0 \quad \text{para todo nodo } n,$$

es decir, todos los nodos de la frontera tienen la misma prioridad y la búsqueda se vuelve no informada.

Por lo tanto, podría tener el mismo **consumo de tiempo** que **TREE-DFS** y el **mismo consumo de memoria** que **TREE-BFS**, ambos exponenciales.

Sin embargo, **con una heurística apropiada, la cantidad de nodos generados puede reducirse drásticamente**. No veremos mayores detalles...

Entre UCS y GBFS

- UCS garantiza optimalidad, pero genera todos los nodos con costo de camino menor al de una solución óptima.
- GBFS puede reducir drásticamente la cantidad de nodos generados con una función heurística apropiada, pero no garantiza optimalidad.

¿Será posible
combinar ambas? 🤔

Búsqueda A*

A* Search

Combina las estrategias **UCS** y **GBFS**.

Elige el nodo a expandir según una combinación de su **costo de camino** y su **valor heurístico**.

Se define una **función de evaluación** f tal que, dado un nodo n ,

$$f(n) = n.\text{costo} + h(n)$$

es decir, $f(n)$ es una **estimación del costo de una solución que pasa por el estado de n** .

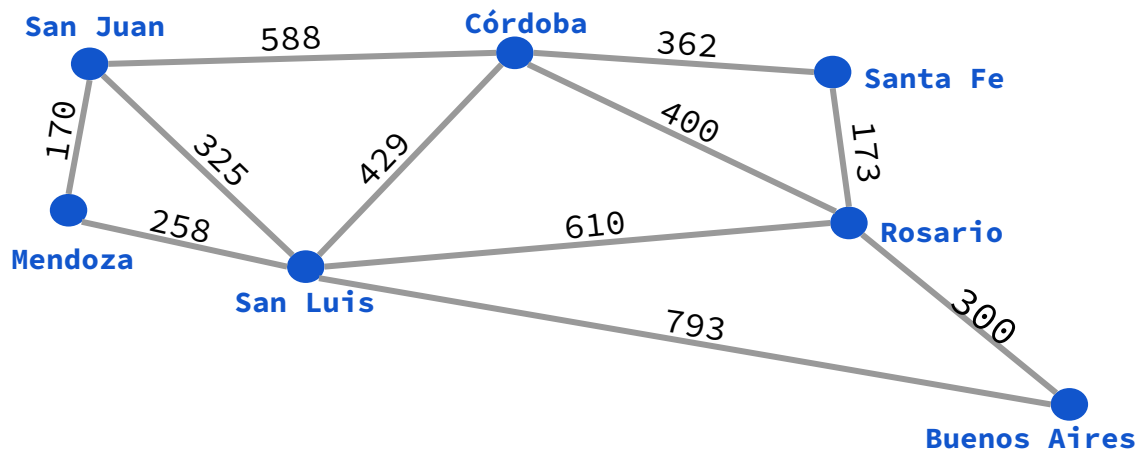
El nodo n que se extrae de la frontera es siempre el de **menor valor de evaluación $f(n)$** .

— — —



Ejemplo – Problema de camino más corto

Vamos a resolver el problema de camino más corto con la búsqueda de árbol usando la estrategia de búsqueda A*.



Distancia en línea recta a San Juan desde:

Buenos Aires	= 997
Rosario	= 747
Santa Fe	= 743
Córdoba	= 417
San Luis	= 284
Mendoza	= 148
San Juan	= 0



Ejemplo

— — —

Anotamos encima de cada nodo su **costo de camino**, su **valor heurístico** y su **valor de evaluación**.

$$0 + 997 = 997$$

n_0 :
Bs. As.
0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			



Ejemplo

— — —

Extraemos el único nodo de la frontera y lo expandimos.

$$0 + 997 = 997$$

n_0 :
Bs. As.
0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			



Ejemplo

— — —

$$0 + 997 = 997$$

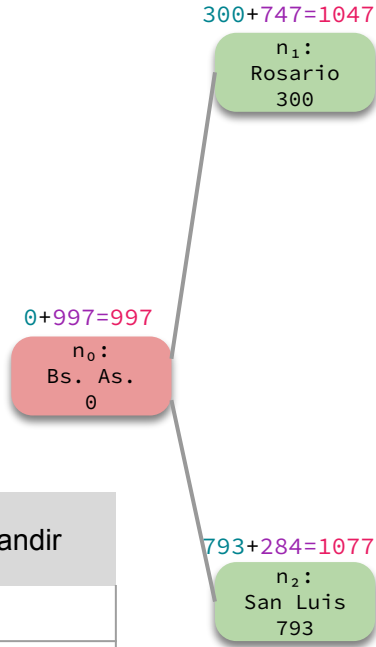
n_0 :
Bs. As.
0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			
5			



Ejemplo

— — —

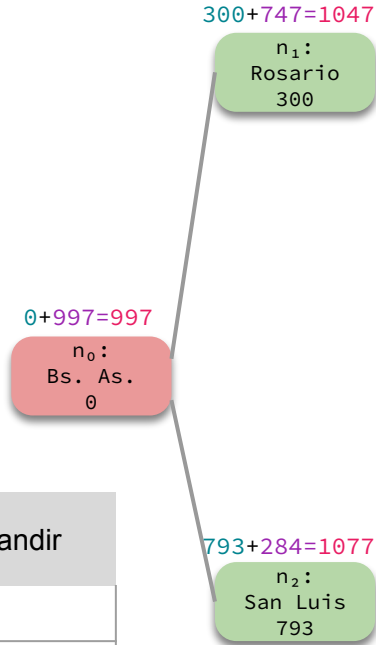


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	{ n_1, n_2 }
2			
3			
4			
5			



Ejemplo

— — —



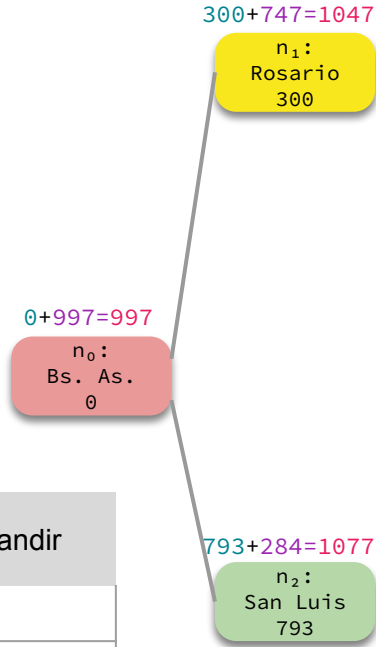
La frontera tiene 2 nodos, extraemos el de **menor valor de evaluación**, es decir a n_1 .

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			
4			
5			



Ejemplo

— — —

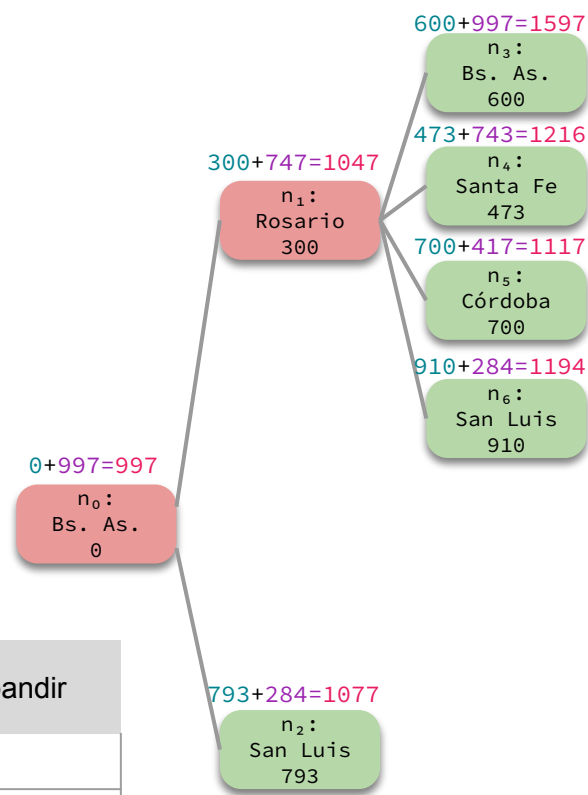


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	
3			
4			
5			



Ejemplo

— — —

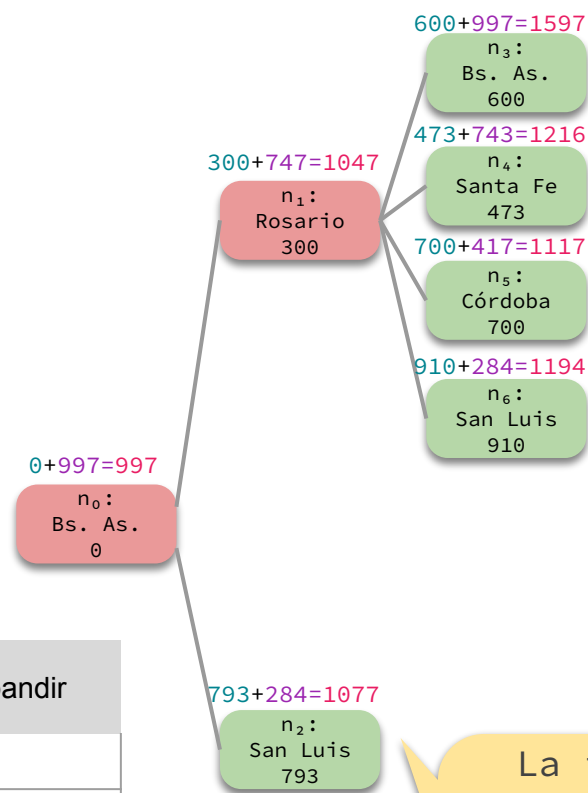


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3			
4			
5			



Ejemplo

— — —



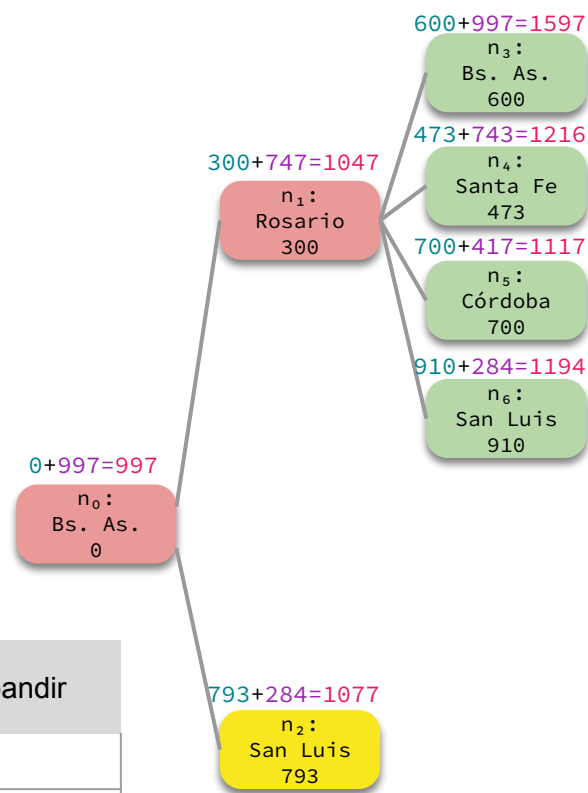
La frontera tiene 5 nodos, extraemos el de **menor valor de evaluación**, es decir a n_2 .

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3, n_4, n_5, n_6\}$
3			
4			
5			



Ejemplo

— — —



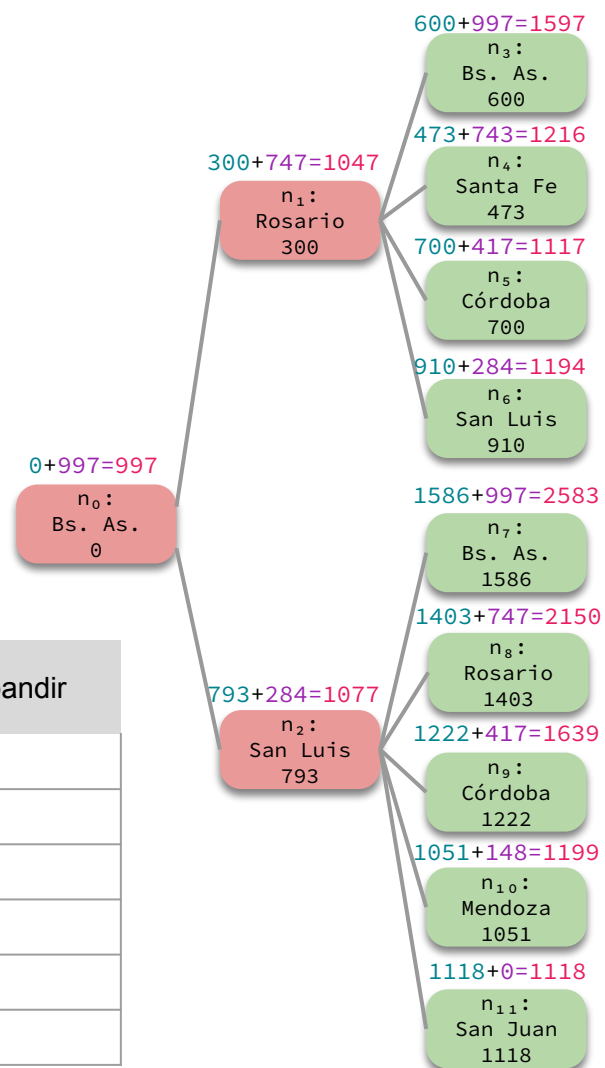
Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₂	{n ₃ , n ₄ , n ₅ , n ₆ }	
4			
5			



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₂	{n ₃ , n ₄ , n ₅ , n ₆ }	{n ₃ , n ₄ , n ₅ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }
4			
5			

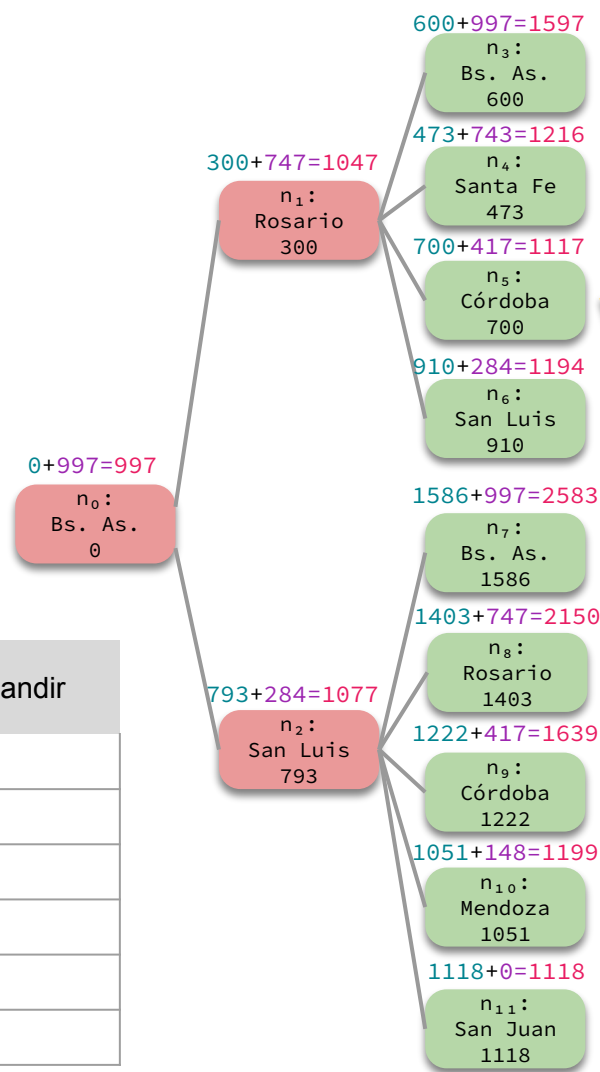




Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₂	{n ₃ , n ₄ , n ₅ , n ₆ }	{n ₃ , n ₄ , n ₅ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }
4			
5			



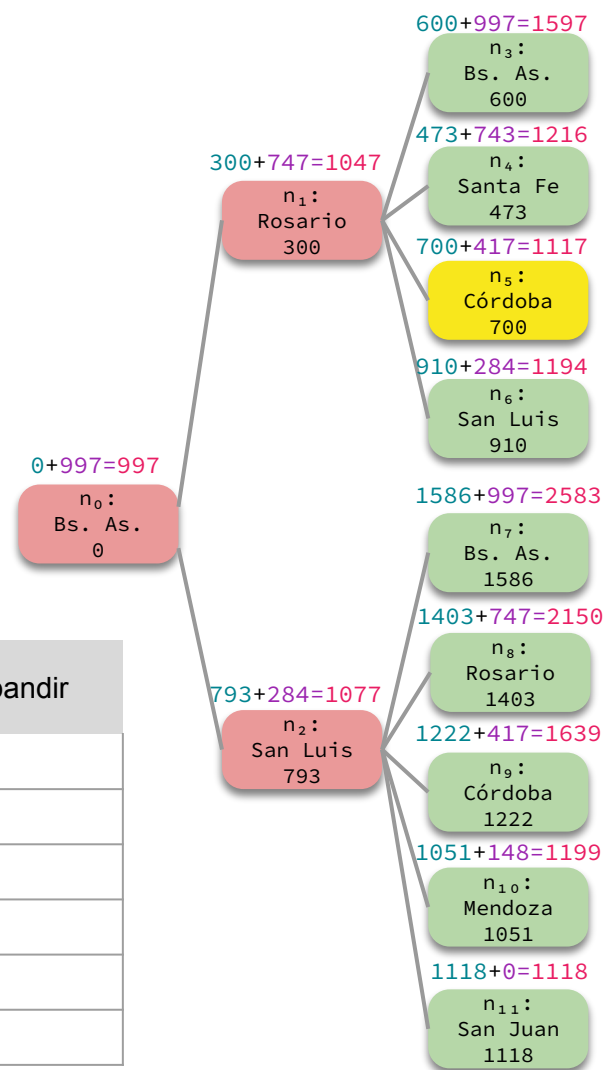
La frontera tiene 9 nodos, extraemos el de **menor valor de evaluación**, es decir a n_5 .



Ejemplo

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₂	{n ₃ , n ₄ , n ₅ , n ₆ }	{n ₃ , n ₄ , n ₅ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }
4	n ₅	{n ₃ , n ₄ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }	
5			

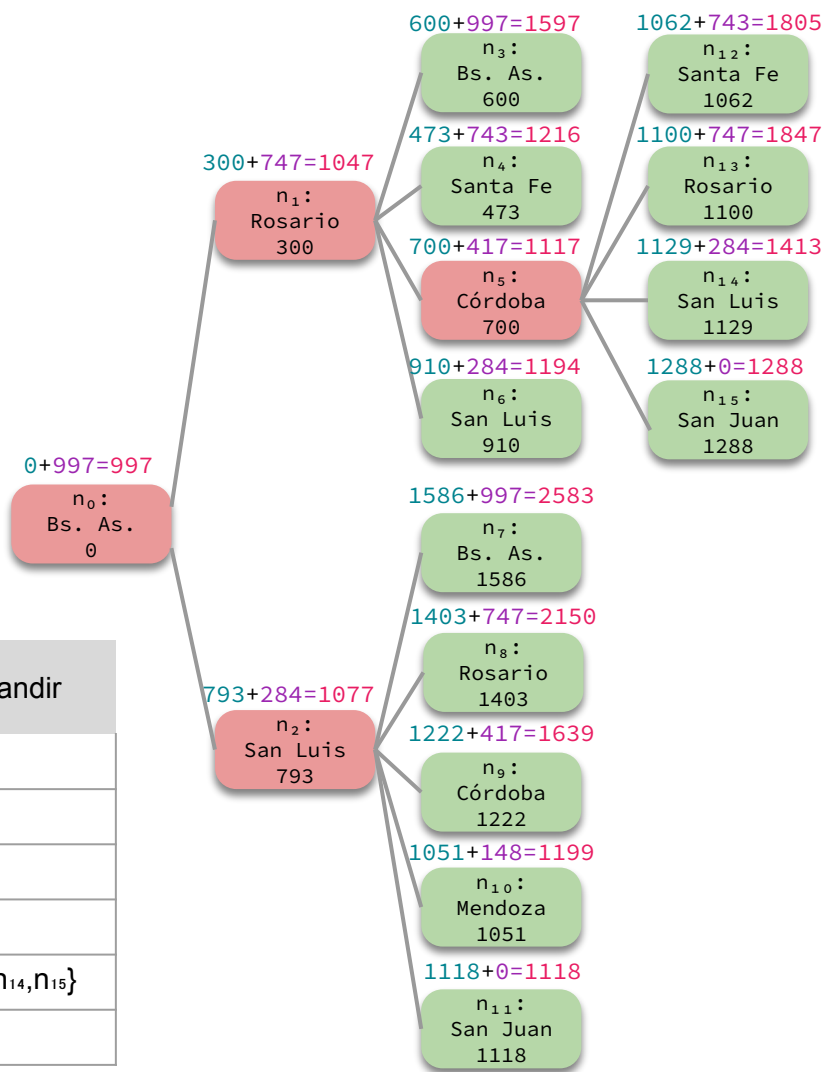




Ejemplo

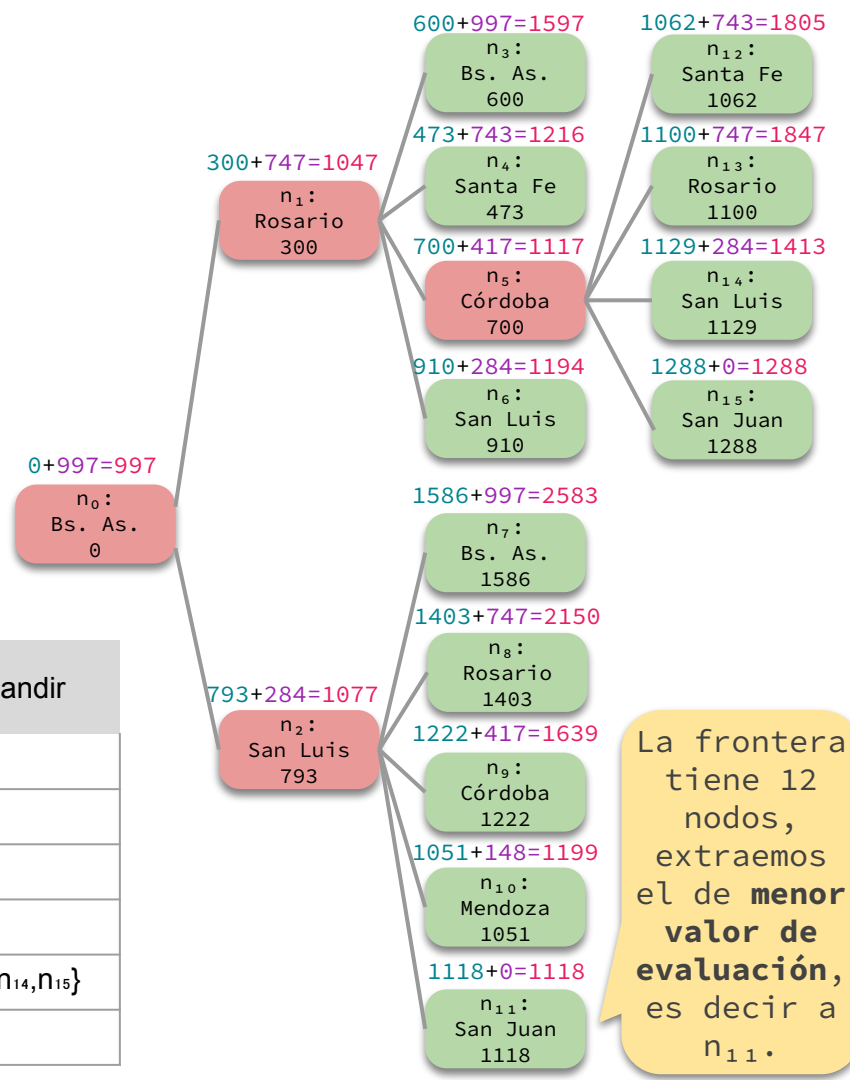
— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₂	{n ₃ , n ₄ , n ₅ , n ₆ }	{n ₃ , n ₄ , n ₅ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }
4	n ₅	{n ₃ , n ₄ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }	{n ₃ , n ₄ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ , n ₁₂ , n ₁₃ , n ₁₄ , n ₁₅ }
5			





Ejemplo

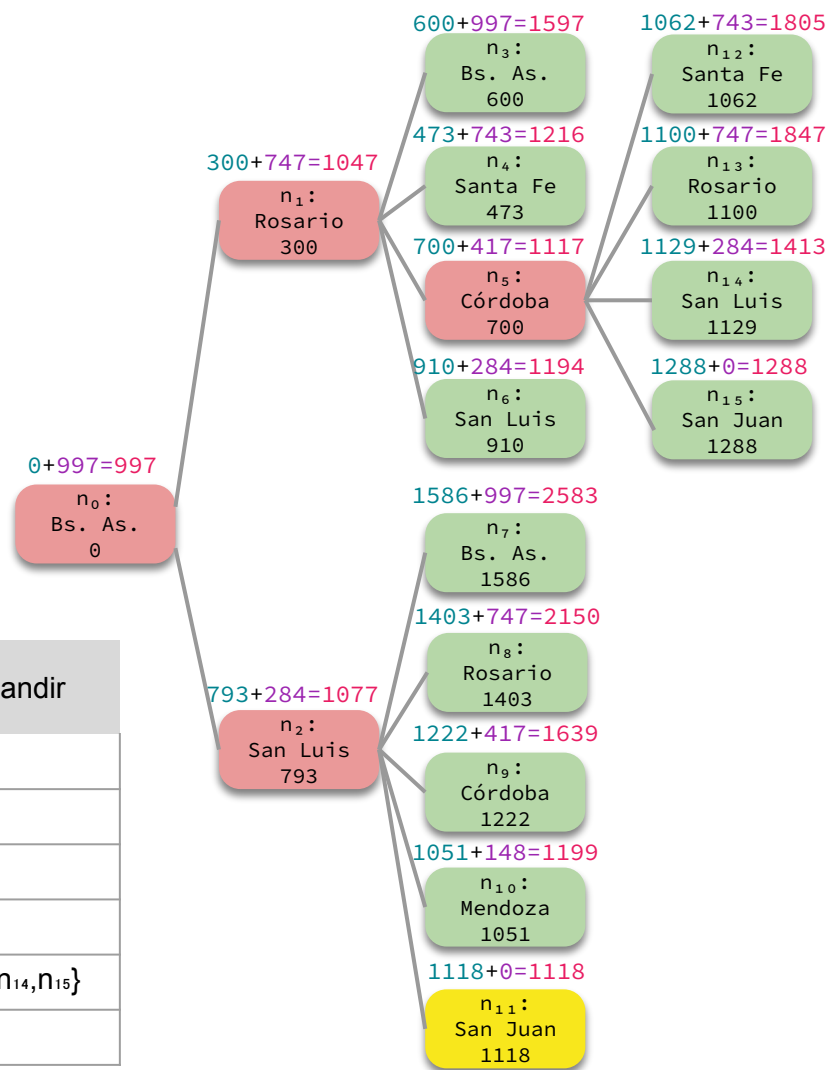


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	{ n_1, n_2 }
2	n_1	{ n_2 }	{ n_2, n_3, n_4, n_5, n_6 }
3	n_2	{ n_3, n_4, n_5, n_6 }	{ $n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }
4	n_5	{ $n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }	{ $n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}, n_{13}, n_{14}, n_{15}$ }
5			



Ejemplo

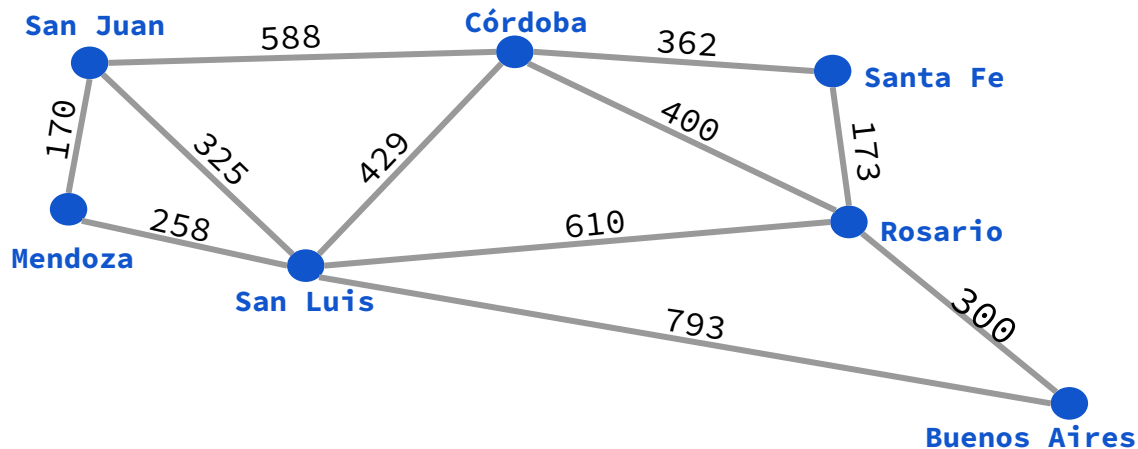
Ejecutamos el test objetivo
y retornamos la solución.
Encontramos solución **óptima**.
¿Será siempre así? 🤔



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	{ n_1, n_2 }
2	n_1	{ n_2 }	{ n_2, n_3, n_4, n_5, n_6 }
3	n_2	{ n_3, n_4, n_5, n_6 }	{ $n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }
4	n_5	{ $n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }	{ $n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}, n_{13}, n_{14}, n_{15}$ }
5	n_{11}	{ $n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{12}, n_{13}, n_{14}, n_{15}$ }	¡FIN!

Función heurística

¿Qué pasaría si tipeamos mal la distancia de “San Luis” a “San Juan”? Vamos a resolver el problema de camino más corto con la búsqueda de árbol usando la estrategia de búsqueda A* con la siguiente heurística modificada:



Heurística:

Buenos Aires	=	997
Rosario	=	747
Santa Fe	=	743
Córdoba	=	417
San Luis	=	999
Mendoza	=	148
San Juan	=	0



Ejemplo

$$0 + 997 = 997$$

n_0 :
Bs. As.
0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			
6			



Ejemplo

— — —

$$0 + 997 = 997$$

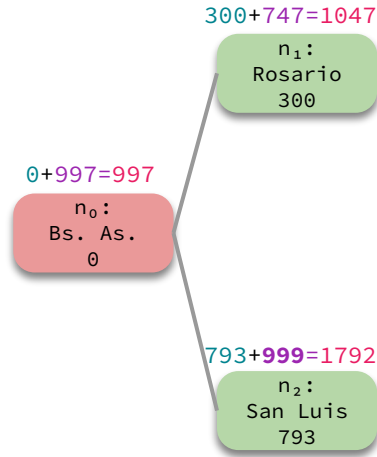
n_0 :
Bs. As.
0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			
5			
6			



Ejemplo

— — —

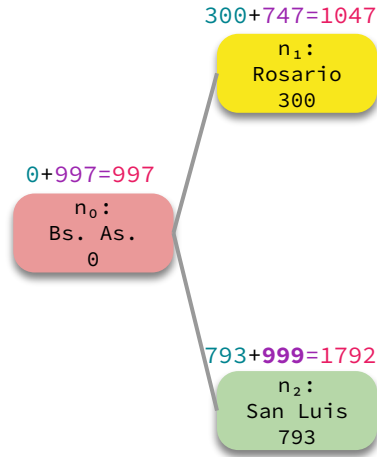


Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			
4			
5			
6			



Ejemplo

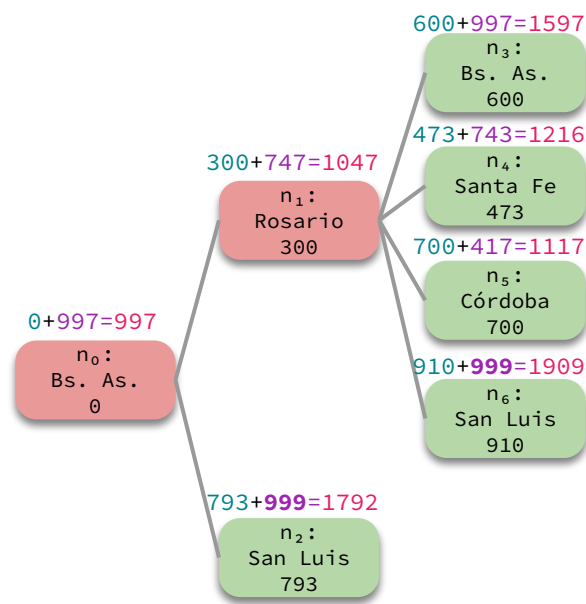
— — —



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	
3			
4			
5			
6			



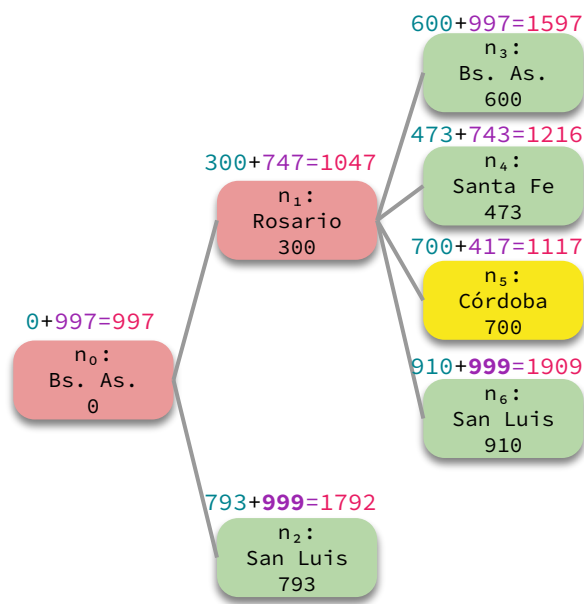
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3, n_4, n_5, n_6\}$
3			
4			
5			
6			



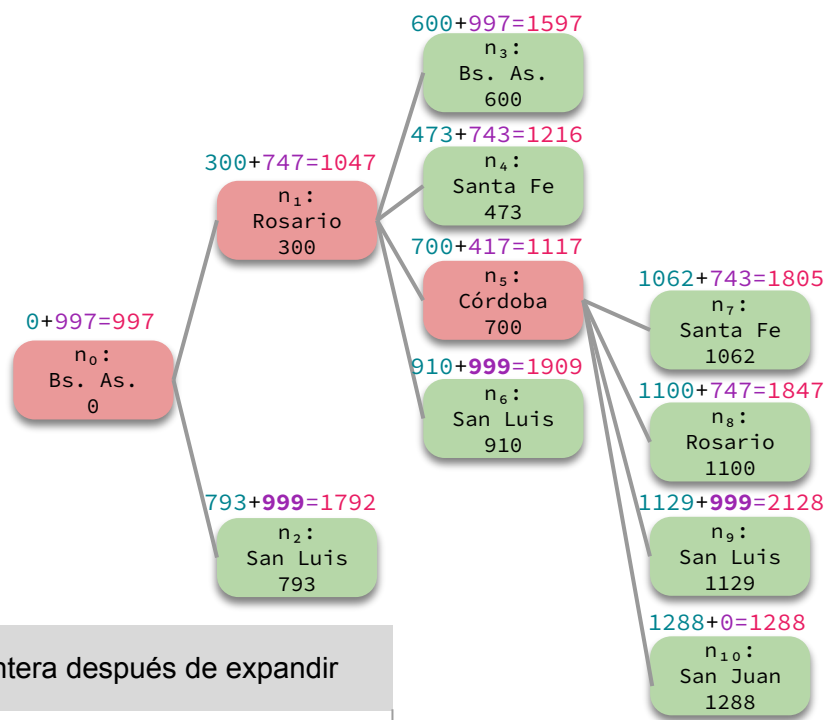
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3, n_4, n_5, n_6\}$
3	n_5	$\{n_2, n_3, n_4, n_6\}$	
4			
5			
6			



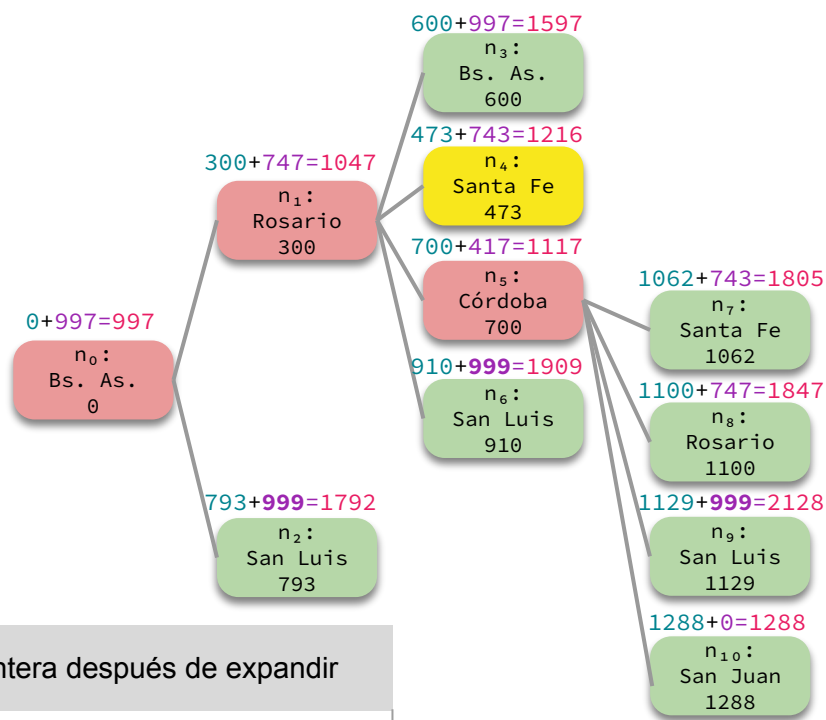
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	{ n_1, n_2 }
2	n_1	{ n_2 }	{ n_2, n_3, n_4, n_5, n_6 }
3	n_5	{ n_2, n_3, n_4, n_6 }	{ $n_2, n_3, n_4, n_6, n_7, n_8, n_9, n_{10}$ }
4			
5			
6			



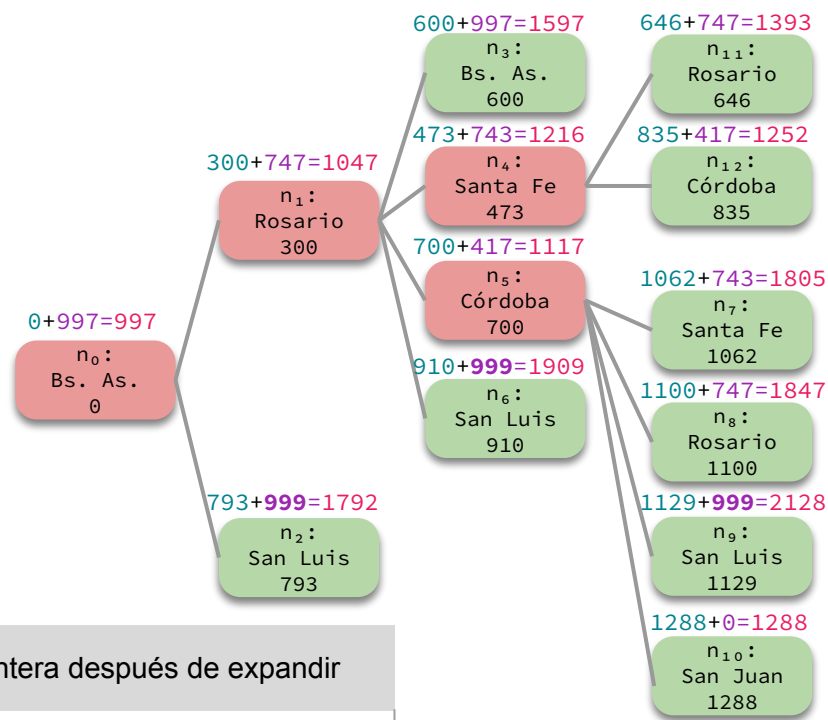
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3, n_4, n_5, n_6\}$
3	n_5	$\{n_2, n_3, n_4, n_6\}$	$\{n_2, n_3, n_4, n_6, n_7, n_8, n_9, n_{10}\}$
4	n_4	$\{n_2, n_3, n_6, n_7, n_8, n_9, n_{10}\}$	
5			
6			



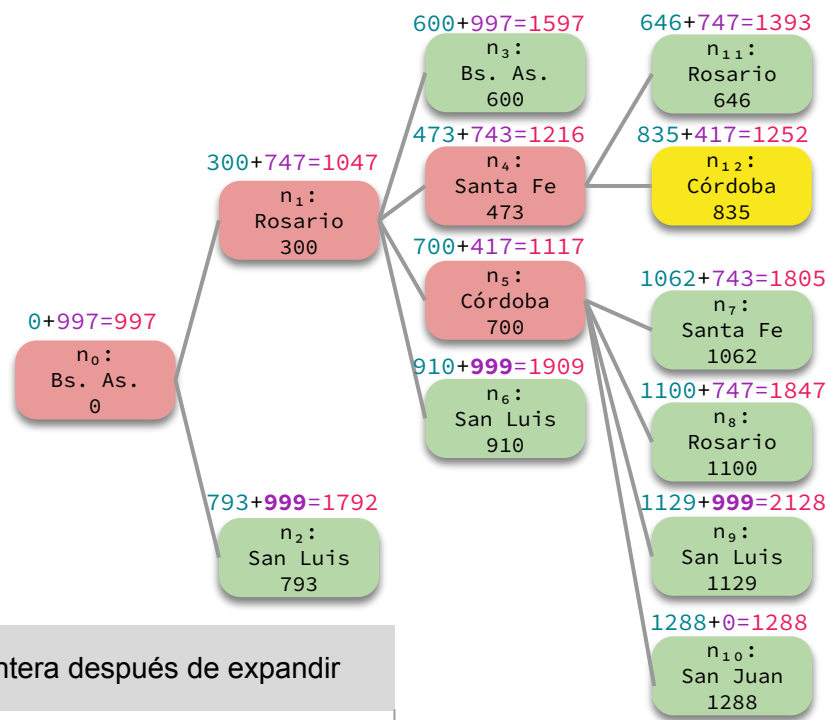
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3, n_4, n_5, n_6\}$
3	n_5	$\{n_2, n_3, n_4, n_6\}$	$\{n_2, n_3, n_4, n_6, n_7, n_8, n_9, n_{10}\}$
4	n_4	$\{n_2, n_3, n_6, n_7, n_8, n_9, n_{10}\}$	$\{n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}\}$
5			
6			



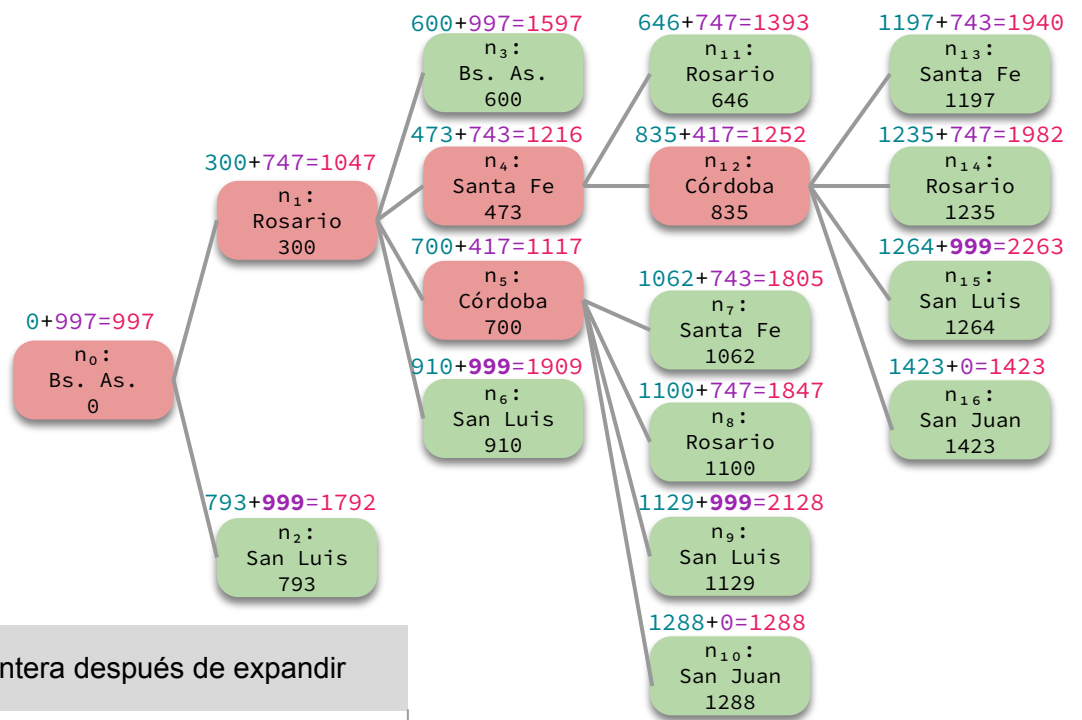
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₁	{n ₂ }	{n ₂ , n ₃ , n ₄ , n ₅ , n ₆ }
3	n ₅	{n ₂ , n ₃ , n ₄ , n ₆ }	{n ₂ , n ₃ , n ₄ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ }
4	n ₄	{n ₂ , n ₃ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ }	{n ₂ , n ₃ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ , n ₁₂ }
5	n ₁₂	{n ₂ , n ₃ , n ₆ , n ₇ , n ₈ , n ₉ , n ₁₀ , n ₁₁ }	
6			



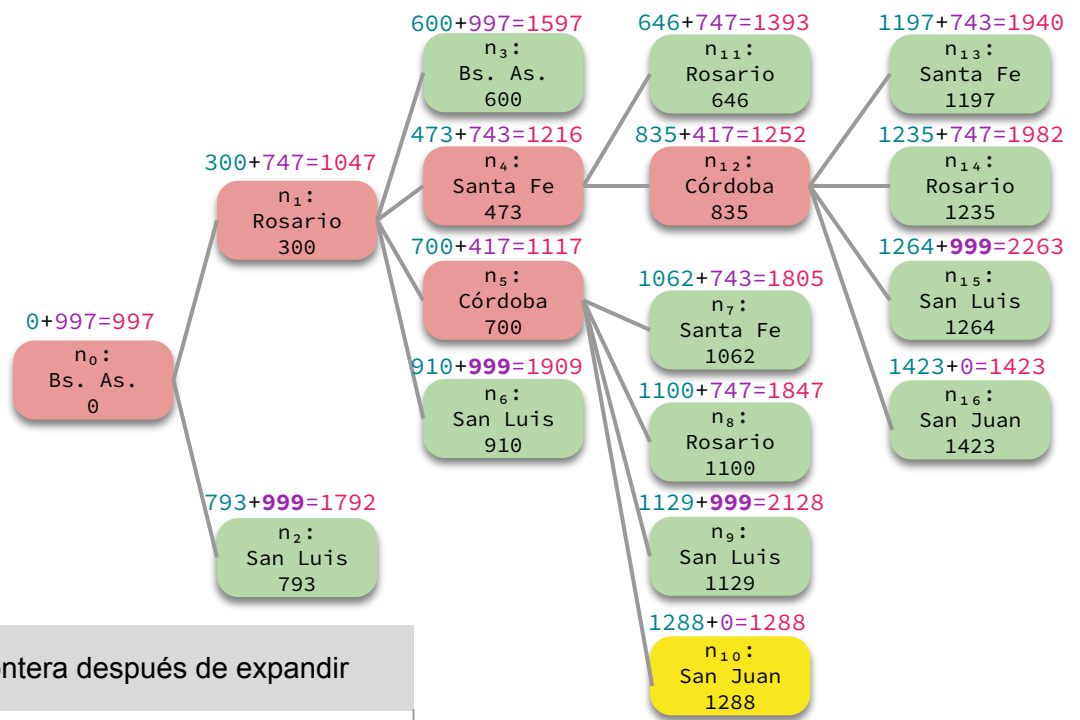
Ejemplo



Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	{ n_1, n_2 }
2	n_1	{ n_2 }	{ n_2, n_3, n_4, n_5, n_6 }
3	n_5	{ n_2, n_3, n_4, n_6 }	{ $n_2, n_3, n_4, n_6, n_7, n_8, n_9, n_{10}$ }
4	n_4	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}$ }	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}$ }
5	n_{12}	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{13}, n_{14}, n_{15}, n_{16}$ }
6			



Ejemplo



En este caso, encontramos una solución **subóptima**. ¿Por qué? 🤔
Lo responderemos en breve...

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{}	{ n_1, n_2 }
2	n_1	{ n_2 }	{ n_2, n_3, n_4, n_5, n_6 }
3	n_5	{ n_2, n_3, n_4, n_6 }	{ $n_2, n_3, n_4, n_6, n_7, n_8, n_9, n_{10}$ }
4	n_4	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}$ }	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}$ }
5	n_{12}	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}$ }	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{13}, n_{14}, n_{15}, n_{16}$ }
6	n_{10}	{ $n_2, n_3, n_6, n_7, n_8, n_9, n_{11}, n_{13}, n_{14}, n_{15}, n_{16}$ }	¡FIN!



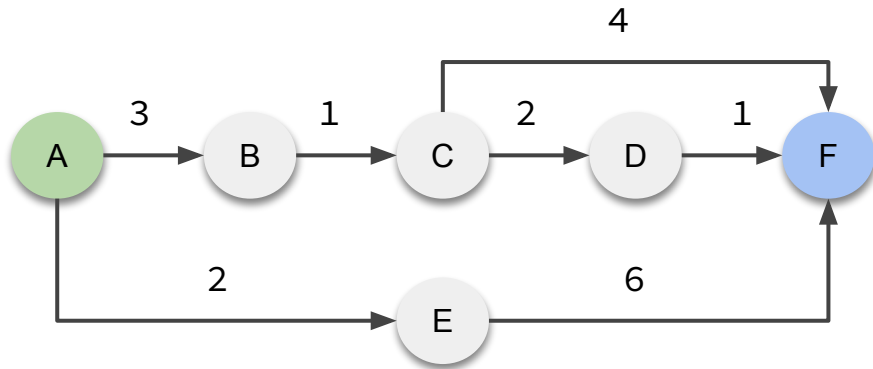
Ejercicio

Sea un problema con el siguiente grafo de espacio de estados, donde A es el estado inicial y F es el estado objetivo.

Resolverlo con el algoritmo general de búsqueda de árbol con la estrategia A*.

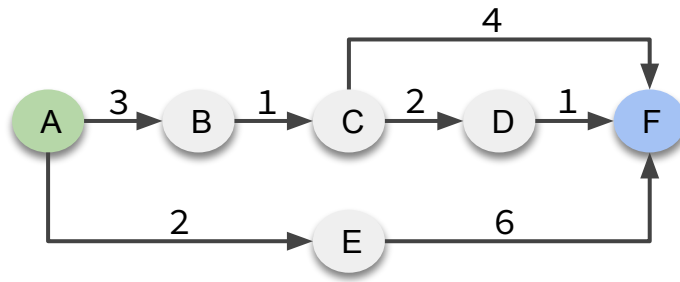
Usar la heurística dada por:

A = 6, B = 3, C = 2, D = 1, E = 5, F = 0





Respuesta



Valores heurísticos:

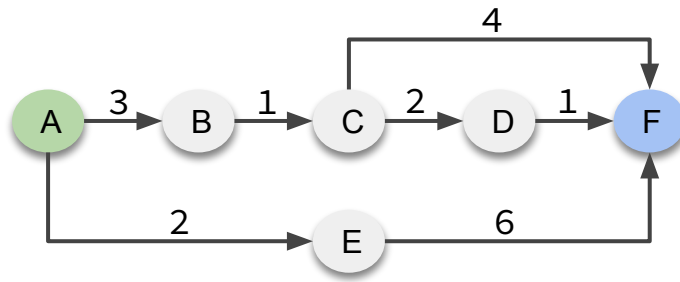
A = 6, B = 3, C = 2,
D = 1, E = 5, F = 0

n_0
A 0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1			
2			
3			
4			
5			



Respuesta



Valores heurísticos:

A = 6, B = 3, C = 2,
D = 1, E = 5, F = 0

n_0
A 0

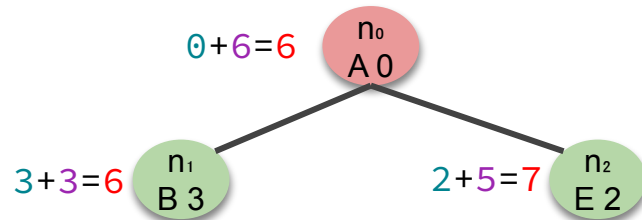
Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{ n_0 }	-
1	n_0	{ }	
2			
3			
4			
5			



Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			
4			
5			

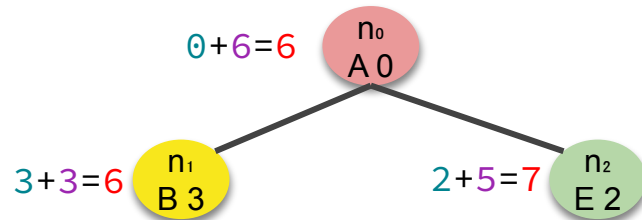




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	
3			
4			
5			

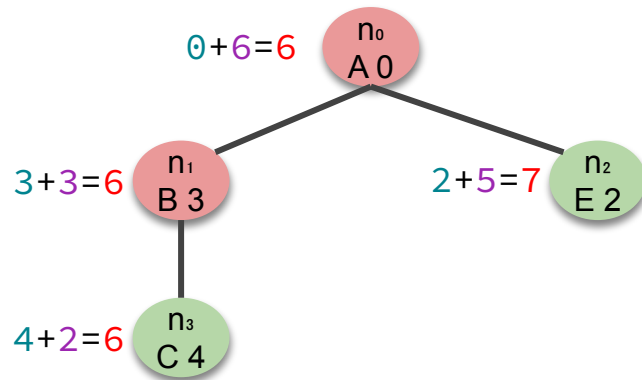




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3			
4			
5			

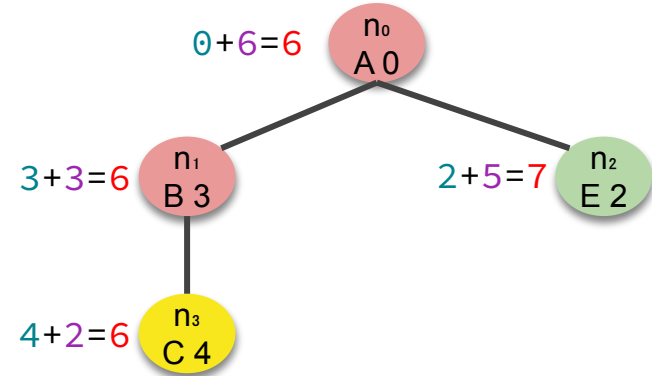




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	
4			
5			

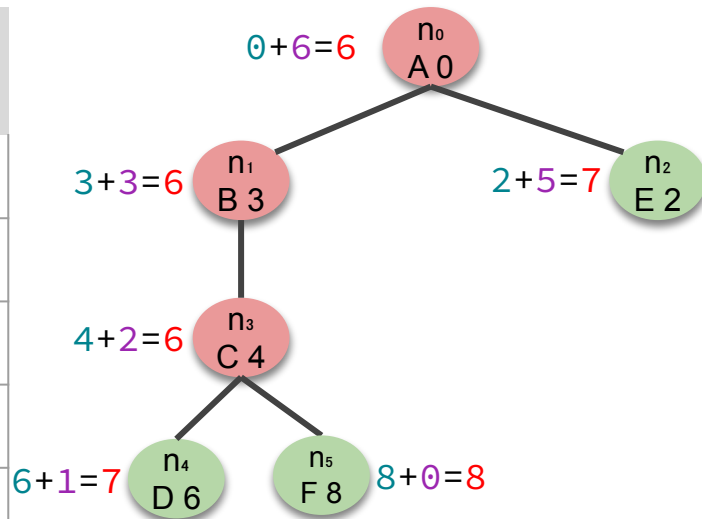




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4			
5			

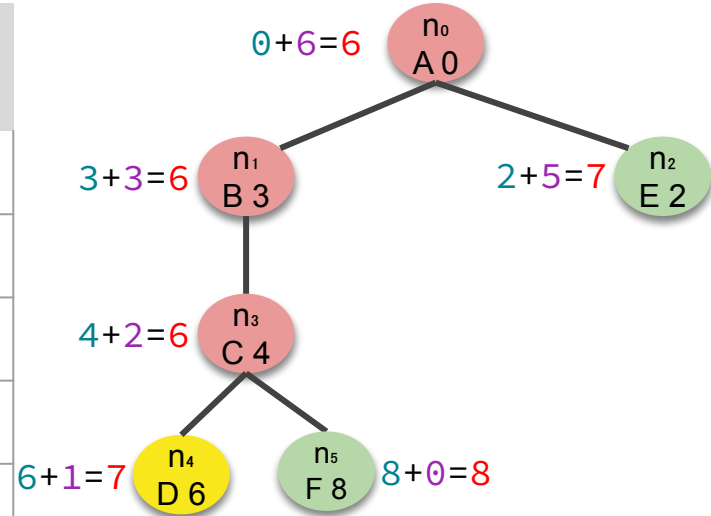




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4	n_4	$\{n_2, n_5\}$	
5			

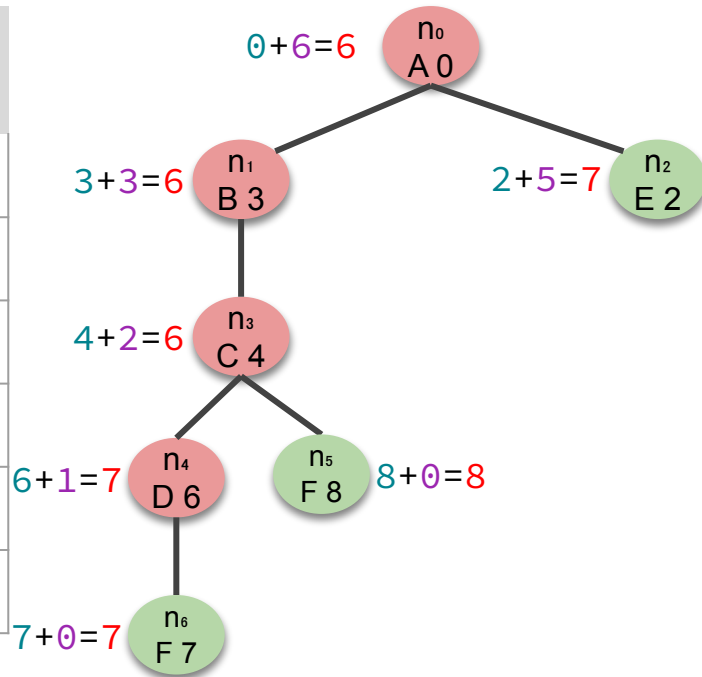




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4	n_4	$\{n_2, n_5\}$	$\{n_2, n_5, n_6\}$
5			

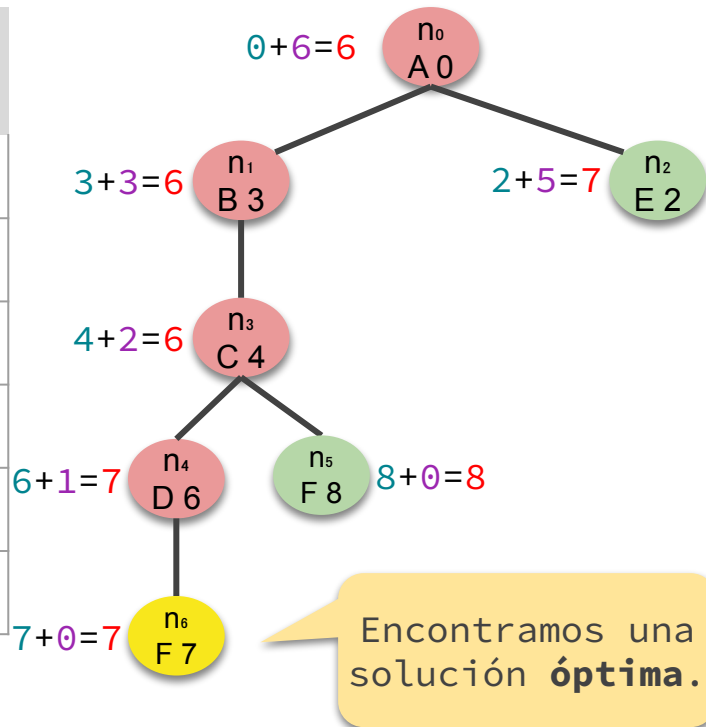




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_1	$\{n_2\}$	$\{n_2, n_3\}$
3	n_3	$\{n_2\}$	$\{n_2, n_4, n_5\}$
4	n_4	$\{n_2, n_5\}$	$\{n_2, n_5, n_6\}$
5	n_6	$\{n_2, n_5\}$	¡Fin!



Implementación

- ¿Cómo elegimos de la frontera el próximo nodo a expandir?

El nodo n con la menor evaluación $f(n)$.

- ¿Cómo lo logramos?

Al igual que en UCS y GBFS, usando el TAD **cola de prioridad** para la frontera pero ordenando los nodos por la función de evaluación f .



Algoritmo A* de grafo

```
1 function GRAPH-ASTAR(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   frontera ← ColaPrioridad()
4   frontera.encolar(n0, n0.costo + problema.h(n0))
5   alcanzados ← {n0.estado: n0.costo}
6   do
7     if frontera.vacía() then return fallo
8     n ← frontera.desencolar()
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      c' ← n.costo + problema.costo-individual(n.estado, a)
13      if s' is not in alcanzados or c' < alcanzados[s'] then
14        n' ← Nodo(s', n, a, c')
15        alcanzados[s'] ← c'
16        frontera.encolar(n', n'.costo + problema.h(n'))
```

Suponemos que el problema tiene un **método** *h* que implementa la heurística.

Al igual que UCS, es importante que el test objetivo se llame al sacar de la frontera.

TREE-ASTAR es análogo. Hay que borrar únicamente las partes relacionadas con el diccionario de alcanzados.

Completitud y optimalidad

La completitud y optimalidad dependen de la heurística.

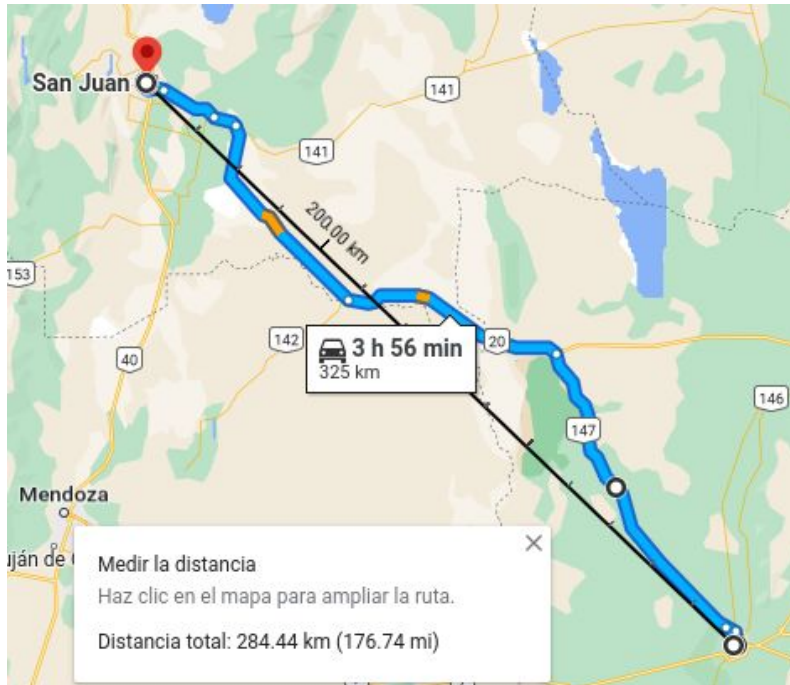
Definición. Una heurística h es **admisibile** si, para todo nodo n , $h(n)$ nunca sobrestima el mínimo costo de camino desde el estado de n a un estado objetivo.

Veamos un ejemplo...



Ejemplo – Problema de camino más corto

— — —



La distancia en ruta de “San Luis” a “San Juan” es COSTO-INDIVIDUAL(San Luis, San Juan) = **325** km, pero la distancia en línea recta es de **284** km.

La distancia más corta entre dos puntos siempre está dado por un **segmento de recta**, pero generalmente las rutas no son rectas (por ejemplo, desvían accidentes geográficos).

Luego, **la distancia en línea recta entre dos ciudades siempre es menor o igual a la menor distancia en ruta entre ellas.**

Por lo tanto, la distancia en línea recta es una heurística **admisable** para el problema de camino más corto.

Completitud y optimalidad

Definición. Una heurística h es **consistente** si, para todo nodo n e hijo n' generado por una acción a , se cumple

$$h(n) \leq \text{COSTO-INDIVIDUAL}(n.\text{estado}, a) + h(n').$$

Propiedad. Toda heurística consistente es admisible.

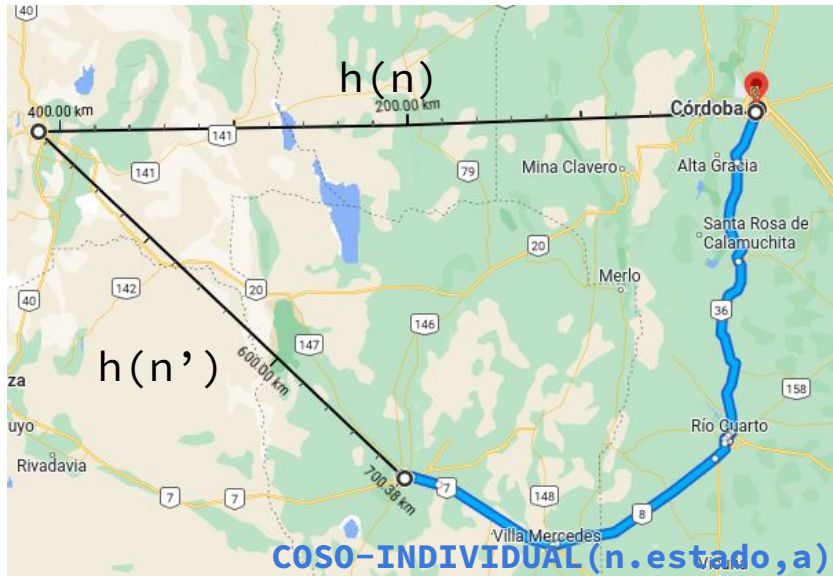
Veamos un ejemplo...

No veremos la demostración.



Ejemplo – Problema de camino más corto

— — —



Considerar

- n : un nodo con el estado “Córdoba”.
- a : acción “San Luis”
- n' : un nodo hijo con el estado “San Luis”.

Luego,

- $\text{COSTO-INDIVIDUAL}(n.\text{estado}, a) = 429$
- $h(n) = 417$ (dist. en línea recta)
- $h(n') = 284$ (dist. en línea recta)

y se tiene

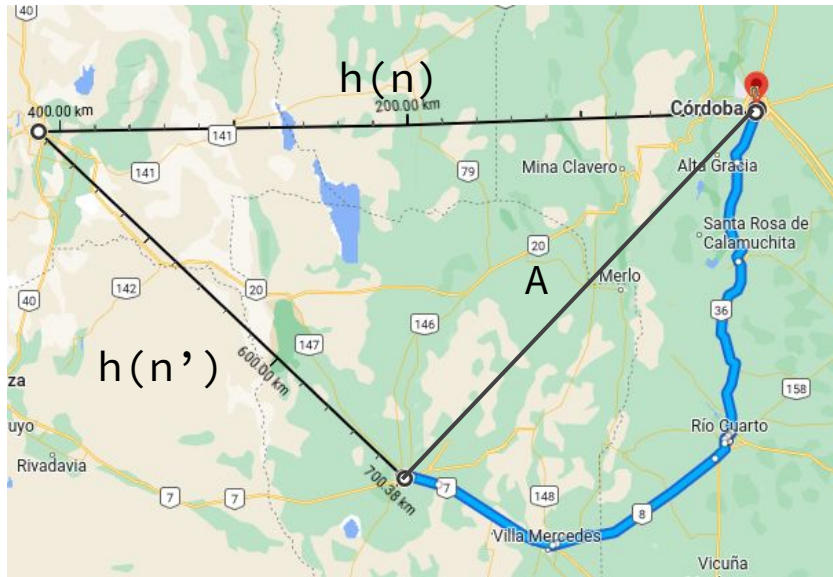
$$h(n) \leq \text{COSTO-INDIVIDUAL}(n.\text{estado}, a) + h(n')$$

$$417 \leq 429 + 284 = 713 \quad \checkmark$$



Ejemplo – Problema de camino más corto

— — —



Para que la heurística sea **consistente**, la desigualdad se tiene que cumplir **para todo** n , a y n' . Pero esto vale, veamos:

Sea A la distancia en línea recta entre n .estado y n' .estado. Por **desigualdad triangular**:

$$h(n) \leq A + h(n')$$

Y por lo que dijimos anteriormente,

$$A \leq \text{COSTO-INDIVIDUAL}(n.\text{estado}, a)$$

Por lo tanto, se cumple

$$h(n) \leq \text{COSTO-INDIVIDUAL}(n.\text{estado}, a) + h(n')$$

Completitud y optimalidad

Teorema.

- **TREE-ASTAR** garantiza completitud y optimalidad si la heurística h es **admisibile**.
- **GRAPH-ASTAR** garantiza completitud y optimalidad si la heurística h es **consistente**.

Demostración. No la vamos a ver.

Consumo de tiempo y memoria

El consumo de tiempo y memoria de **TREE-ASTAR** es difícil de medir porque depende de la heurística. Por ejemplo, si la heurística es

$$h(n) = 0 \quad \text{para todo nodo } n,$$

entonces es equivalente a **TREE-UCS**, y por lo tanto tiene el mismo consumo de tiempo y memoria.

A su vez, en problemas con muchos caminos redundantes, **GRAPH-ASTAR** puede reducir drásticamente el consumo de tiempo y memoria respecto a **TREE-ASTAR**.



Resumen

- Vimos dos estrategias de búsqueda informadas: GBFS y A*.
- GBFS expande siempre el nodo no expandido con menor costo heurístico $h(n)$. No garantiza optimalidad pero suele ser bastante eficiente.
- A* expande siempre el nodo no expandido con menor costo de evaluación $f(n) = n.\text{costo} + h(n)$. Garantiza optimalidad si la heurística es admisible (búsqueda de árbol) y consistente (búsqueda de grafo).



Próximamente

— — —

¿Cómo construir heurísticas? 🤔